

Universidad
Rey Juan Carlos

Escuela Técnica Superior
de Ingeniería Informática

Grado en Diseño y Desarrollo de Videojuegos

Curso 2025-2026

Trabajo Fin de Grado

**IMPLEMENTACIÓN DE UN ENTORNO DE RV
CONSTRUIDO MEDIANTE UN ROBOT TURTLEBOT**

Autor: Rodrigo Hervada Llahosa

Tutor: Jonathan Crespo Herrero

Agradecimientos

Agradezco a todas las personas que han contribuido, por acción o por omisión, a la realización de este trabajo.

Resumen

Este trabajo presenta el diseño e implementación de un sistema de reconstrucción y visualización tridimensional del entorno de un robot móvil Turtlebot2 integrado en una experiencia de realidad virtual desarrollada en Unity. El sistema permite generar automáticamente un entorno virtual navegable a partir de los datos sensoriales del robot, representarlo mediante cuatro técnicas distintas de visualización tridimensional, y utilizarlo como escenario para un minijuego cooperativo en el que el operador y el robot deben coordinarse para alcanzar objetivos comunes.

La arquitectura del sistema integra tres componentes principales: el stack ROS Kinetic ejecutado sobre el Turtlebot2 con Ubuntu 16.04, una aplicación de realidad virtual desarrollada en Unity 2022 que actúa como núcleo del sistema, y las gafas Meta Quest 2 como dispositivo de interacción inmersiva. La comunicación entre Unity y ROS se realiza mediante el protocolo ROSBridge a través de WebSocket usando el paquete ROS#. Las representaciones del entorno implementadas comprenden un mapa de paredes generado a partir del mapa de ocupación 2D, una malla poligonal procedimental, un sistema de partículas y una representación volumétrica mediante vóxeles basada en OctoMap, todas ellas seleccionables en tiempo de ejecución. El sistema incluye además persistencia de mapas en disco, lo que permite trabajar con mapas preconstruidos sin necesidad de conexión al robot.

El proyecto demuestra que es posible construir una experiencia inmersiva y gamificada sobre una plataforma robótica de bajo coste y software legacy, utilizando exclusivamente herramientas de acceso libre.

Abstract

This project presents the design and implementation of a three-dimensional reconstruction and visualization system of the environment of a Turtlebot2 mobile robot integrated into a virtual reality experience developed in Unity. The system allows automatically generating a navigable virtual environment from the robot’s sensory data, representing it using four different three-dimensional visualization techniques, and using it as a scenario for a cooperative mini-game in which the operator and the robot must coordinate to achieve common objectives.

The system architecture integrates three main components: the ROS Kinetic stack running on the Turtlebot2 with Ubuntu 16.04, a virtual reality application developed in Unity 2022 that acts as the core of the system, and the Meta Quest 2 headset as an immersive interaction device. Communication between Unity and ROS is carried out through the ROSBridge protocol via WebSocket using the ROS# package. The implemented environment representations include a wall map generated from the 2D occupancy grid map, a procedural polygonal mesh, a particle system, and a volumetric representation using voxels based on OctoMap, all of them selectable at runtime. The system also includes map persistence on disk, which makes it possible to work with prebuilt maps without requiring a connection to the robot.

The project demonstrates that it is possible to build an immersive and gamified experience on a low-cost robotic platform with legacy software, using exclusively open-access tools.

Acrónimos y abreviaturas

- **ROS**: Robot Operating System (Sistema Operativo de Robots)
- **VR**: Virtual Reality (Realidad Virtual)
- **ROS#**: ROS Sharp (Paquete para la comunicación entre Unity y ROS)
- **ROSbridge**: ROSbridge Protocol (Protocolo para la comunicación entre ROS y otras aplicaciones)
- **API**: Application Programming Interface (Interfaz de Programación de Aplicaciones)
- **SLAM**: Simultaneous Localization and Mapping (Localización y Mapeo Simultáneos)
- **HMD**: Head-Mounted Display (Pantalla Montada en la Cabeza)
- **RGB-D**: Red Green Blue - Depth (Sensor de color y profundidad)
- **URDF**: Unified Robot Description Format (Formato Unificado de Descripción de Robots)
- **TF**: Transform (Sistema de transformaciones de coordenadas en ROS)
- **FPS**: Frames Per Second (Fotogramas por segundo)
- **GPU**: Graphics Processing Unit (Unidad de Procesamiento Gráfico)
- **CPU**: Central Processing Unit (Unidad Central de Procesamiento)
- **PCL**: Point Cloud Library (Librería de Nubes de Puntos)
- **UDP**: User Datagram Protocol (Protocolo de Datagramas de Usuario)
- **JSON**: JavaScript Object Notation (Formato de intercambio de datos)
- **POV**: Point of View (Punto de vista)
- **XR**: Extended Reality (Realidad Extendida)
- **FLU**: Forward, Left, Up (Adelante, Izquierda, Arriba)

Índice de contenidos

Índice de tablas	IX
Índice de figuras	XI
1. Introducción	1
1.1. Motivación	1
1.2. Objetivo general	2
1.2.1. Objetivos específicos	2
1.3. Estructura del documento	3
1.4. Metodología y plan de trabajo	4
1.4.1. Cronograma	5
1.4.2. Explicación por tarea	5
2. Estado del arte	7
2.1. Integración ROS-Unity para visualización 3D	7
2.2. Representación tridimensional del entorno en robótica	8
2.3. Técnicas de renderizado eficiente en tiempo real	10
2.4. Sistemas de persistencia de mapas robóticos	11
2.5. Gemelos digitales y reconstrucción de espacios reales	11
2.6. Entornos 3D reales en videojuegos y experiencias interactivas	13
2.7. Comparativa y posicionamiento del proyecto	14
3. Diseño	15
3.1. Diseño del sistema	15
3.2. Criterios de diseño	16
3.3. Decisiones arquitectónicas	16
3.3.1. Topología centralizada	17
3.3.2. Separación de renderizado y comunicaciones	17
3.3.3. Desacoplamiento de las plataformas físicas	17
3.4. Flujo de interacción	17
3.5. Presupuesto y costes	18
4. Desarrollo	20

4.1.	Plataforma de desarrollo	20
4.1.1.	Hardware	20
4.1.2.	Software	21
4.1.3.	Herramientas adicionales	21
4.2.	Arquitectura general del sistema	22
4.3.	Entorno ROS en el Turtlebot2	22
4.3.1.	Stack de software en el robot	22
4.3.2.	Nodo de decimación de nube de puntos <code>pointcloud_relay.py</code>	23
4.3.3.	Nodo de mapa volumétrico <code>octomap_server</code>	24
4.4.	Comunicación con ROS y recepción de datos	24
4.4.1.	Conexión Unity y ROS mediante ROS#	24
4.4.2.	Suscriptor del mapa de ocupación 2D	24
4.4.3.	Recepción de nubes de puntos	25
4.4.4.	Detector de movimiento del robot	25
4.4.5.	Gestión de transformadas temporales	26
4.5.	Representación tridimensional del entorno	27
4.5.1.	Wall Map	27
4.5.2.	Mesh Map	28
4.5.3.	Particle Map	29
4.5.4.	OctoMap	29
4.5.5.	Comparativa de representaciones	30
4.5.6.	Gestor de mapas	31
4.6.	Persistencia de mapas	31
4.7.	Interacción y teleoperación	32
4.7.1.	Entorno VR y jugador	32
4.7.2.	Sistema de puntos de vista	33
4.7.3.	Teleoperación del robot	33
4.8.	Minijuego cooperativo	34
4.9.	Problemas encontrados durante el desarrollo	35
4.9.1.	Obsolescencia de la plataforma.	35
4.9.2.	Incidencia en RosSharp.	35
4.9.3.	Condiciones de carrera entre hilos.	36
4.9.4.	Acumulación ilimitada de vóxeles.	36
4.9.5.	Desalineaciones en la reconstrucción tridimensional	36
4.9.6.	Acceso limitado al robot físico.	36
4.9.7.	Condiciones en el entorno real.	37
5.	Experimentos y métricas	38
5.1.	Metodología experimental	38
5.2.	Parámetros de configuración del sistema	39
5.3.	Resultados	40
5.3.1.	Rendimiento por tipo de mapa	40

5.3.2.	Fidelidad de la reconstrucción	41
5.3.3.	Guardado y carga de mapas	43
5.3.4.	Minijuego	43
6.	Conclusiones y trabajos futuros	44
6.1.	Discusión de resultados	44
6.2.	Cumplimiento de objetivos	45
6.2.1.	Objetivo general	45
6.2.2.	Objetivos específicos	45
6.3.	Conclusiones generales	46
6.4.	Trabajos futuros y posibles mejoras	47
	Bibliografía	48

Índice de tablas

3.1. Desglose de recursos materiales, valor comercial y coste real. . . .	19
3.2. Desglose de los costes de personal.	19
4.1. Comparativa de los cuatro modos de representación del entorno. .	31
5.1. Parámetros de configuración del módulo de mapas.	40

Índice de figuras

3.1.	Diagrama de la arquitectura del sistema.	16
4.1.	*	26
4.2.	*	26
4.3.	Comparativa espacial cenital que demuestra la correcta sincronización y transformación de coordenadas entre el ecosistema ROS y el motor Unity.	26
4.4.	Perspectivas del entorno generado mediante la técnica Wall Map, extrayendo la geometría directamente de la cuadrícula de ocupación 2D.	27
4.5.	*	28
4.6.	*	28
4.7.	Capturas del entorno virtual recreado dinámicamente mediante mallas poligonales (Mesh Map).	28
4.8.	*	29
4.9.	*	29
4.10.	Representación en Unity de la nube de puntos recibida mediante el sistema de mapas de partículas (Particle Map).	29
4.11.	*	30
4.12.	*	30
4.13.	Reconstrucción tridimensional volumétrica basada en la técnica OctoMap dentro del motor Unity.	30
4.14.	*	33
4.15.	*	33
4.16.	Interfaz de usuario integrada en el entorno inmersivo para la gestión de los mapas tridimensionales y el inicio del minijuego.	33
4.17.	Elementos visuales correspondientes a los objetivos y zonas de transferencia de energía en el nivel 4 del minijuego cooperativo.	35
5.1.	Entorno de simulación en Gazebo utilizado para la captura de métricas bajo condiciones de red y entorno controladas.	39
5.2.	Fotografía real del laboratorio utilizado para las pruebas.	41
5.3.	*	42
5.4.	*	42

5.5.	*		42
5.6.	*		42
5.7.	Comparativa directa de la reconstrucción tridimensional. Representación visual generada por los cuatro modelos implementados.		42

1

Introducción

En este capítulo se expone el marco general del trabajo y se abordan los factores que han impulsado su desarrollo. Además, se detallan tanto el objetivo principal como los objetivos específicos definidos, la organización del documento y, finalmente, la metodología utilizada junto con el plan de trabajo seguido para la ejecución del proyecto.

1.1. Motivación

En este proyecto se presenta un sistema de visualización tridimensional virtual del entorno real de un robot móvil Turtlebot2, con una capa de gamificación integrada en un entorno de realidad virtual desarrollado en Unity. El sistema permite al usuario no solo percibir el espacio que rodea al robot mediante distintas representaciones tridimensionales del mapa, sino también, participar en tareas colaborativas con el robot a través de un minijuego diseñado para explorar las posibilidades de interacción entre humano y máquina en entornos inmersivos.

La reconstrucción del entorno en tres dimensiones a partir de datos de sensores reales es una de las funcionalidades más interesantes de la robótica, y apenas explorada para la creación de experiencias inmersivas. Los mapas de ocupación 2D clásicos, aunque útiles para navegación, ofrecen una representación plana del entorno que limita la comprensión espacial. Disponer de una representación tridimensional del espacio, ya sea mediante nubes de puntos coloreadas o mapas de ocupación volumétricos, permite al usuario comprender mejor la geometría del entorno, los obstáculos presentes y las dimensiones reales del espacio explorado.

Por otra parte, la gamificación de sistemas robóticos abre una línea de trabajo con aplicaciones tanto en educación como en investigación. Diseñar tareas colaborativas entre un operador humano y un robot, estructuradas como niveles de un juego, permite estudiar la interacción humano-robot o incluso diseñar videojuegos utilizando mapas reales. En este proyecto, el minijuego diseñado plantea objetivos que requieren la coordinación entre el operador, que se mueve libremente por el entorno virtual, y el robot, que debe desplazarse físicamente por el espacio real (también se puede hacer con el robot virtual), introduciendo dinámicas de colaboración que no están presentes en sistemas de teleoperación o videojuegos convencionales.

El Turtlebot2, aunque una plataforma de bajo coste con *software legacy*, dispone de una cámara RGB-D y de un *stack* de navegación con SLAM que genera información suficiente para alimentar los distintos sistemas de visualización implementados. Esto demuestra que es posible construir representaciones tridimensionales ricas y experiencias gamificadas inmersivas sin necesidad de *hardware* especializado ni de alto coste.

1.2. Objetivo general

El objetivo general de este trabajo es el desarrollo de un sistema que genere automáticamente un entorno de realidad virtual navegable a partir de la información sensorial capturada por un robot Turtlebot2, integrando dicho entorno con una representación virtual del propio robot y del usuario, además incluyendo un minijuego utilizando dicho entorno 3D para mostrar las posibilidades del proyecto y la interacción entre el jugador y el robot dentro del espacio construido.

El sistema debe ser capaz de reconstruir la geometría del entorno físico en tiempo real a partir de los datos de la cámara RGB-D y el mapa de ocupación 2D generado por SLAM, representarlos mediante distintas técnicas de visualización tridimensional dentro de Unity, e incorporar sobre ese entorno diferentes componentes de videojuegos, como el robot virtual, el jugador y la dinámica de juego entre ambos.

1.2.1. Objetivos específicos

Para lograr el objetivo general, se describen los siguientes objetivos específicos:

- Integrar los datos del mapa de ocupación 2D generado por gmapping en Unity para obtener la geometría del entorno y representarla como un conjunto de paredes navegables en el espacio virtual.

- Implementar y comparar múltiples técnicas de reconstrucción tridimensional del entorno a partir de la nube de puntos RGB-D del robot, incluyendo representación mediante una malla poligonal (*MeshMap*), el sistema de partículas de Unity (*ParticleMap*) y voxeles (*OctoMap*), evaluando sus diferencias en rendimiento y calidad visual.
- Incorporar el modelo URDF del Turtlebot2 en el entorno virtual, sincronizando su posición y orientación con la odometría del robot físico, de forma que el robot virtual se desplace por el entorno generado reflejando el movimiento real del robot.
- Diseñar e implementar un minijuego cooperativo en el que el jugador, controlado mediante las gafas de realidad virtual, y el robot físico deban coordinarse para alcanzar un objetivo común, implementando en forma de mecánica jugable la dependencia entre ambos agentes.
- Validar el sistema mediante pruebas en entorno simulado con Gazebo y, de forma limitada, sobre el robot físico, evaluando el rendimiento, la coherencia espacial del entorno generado y la jugabilidad del minijuego.

1.3. Estructura del documento

Este documento se organiza en seis capítulos principales, complementados por secciones preliminares y apéndices. El Capítulo 1 presenta la motivación del proyecto, los objetivos general y específicos, la metodología seguida y el plan de trabajo con su respectivo cronograma. El Capítulo 2 recoge el estado del arte, analizando trabajos previos en integración entre ROS y Unity para visualización tridimensional, técnicas de representación del entorno en robótica, optimización de renderizado en tiempo real para realidad virtual, sistemas de persistencia de mapas robóticos, reconstrucción de espacios reales como gemelos digitales, y el uso de entornos tridimensionales reales en videojuegos y experiencias interactivas. El Capítulo 3 describe el diseño general del sistema, incluyendo el diagrama de arquitectura y el análisis de costes. El Capítulo 4 detalla el desarrollo e implementación del sistema, describiendo la plataforma *hardware* y *software* utilizada, la arquitectura general, la comunicación con ROS, los cuatro modos de representación tridimensional del entorno, el sistema de persistencia, el entorno de realidad virtual y la lógica del minijuego cooperativo, junto con los problemas encontrados durante el desarrollo. El Capítulo 5 presenta los experimentos realizados y las métricas evaluadas tanto en el entorno simulado como en el robot físico. El Capítulo 6 recoge las conclusiones del trabajo, el cumplimiento de los objetivos y las posibles líneas de trabajo futuras. Finalmente, se incluye la bibliografía y el apéndice con material adicional.

1.4. Metodología y plan de trabajo

El desarrollo de este proyecto se llevó a cabo siguiendo una metodología iterativa e incremental, abordando de forma progresiva cada uno de los módulos que componen el sistema. Dado que el proyecto combina visualización volumétrica en tiempo real, integración con ROS y diseño de mecánicas de juego en un entorno de realidad virtual, el trabajo se organizó en fases sucesivas de implementación y validación.

La primera etapa estuvo dedicada al estudio del entorno de trabajo y las tecnologías necesarias. Se analizaron los distintos enfoques para utilizar los datos de la cámara RGB-D del robot y sus posibles formas de representarlos de manera tridimensional Unity, incluyendo sistemas de partículas, mallas de vóxeles o simples cubos. También se estudió la integración entre ROS y Unity mediante ROS# para la recepción de mensajes como PointCloud2 y OccupancyGrid, así como el funcionamiento del árbol de transformaciones TF de ROS, necesario para proyectar correctamente los puntos en el espacio del mapa.

Una vez comprendidas las tecnologías, se procedió al desarrollo de los distintos sistemas de visualización del mapa tridimensional. Cada representación, mapa de paredes, mapa de malla, mapa de partículas y mapa de octree, fue implementada y validada de forma independiente antes de integrarse bajo un gestor común que permite alternar entre ellas en tiempo de ejecución. Este enfoque modular facilitó la detección de problemas en cada sistema de forma aislada, sin que los errores de uno afectasen al resto.

Paralelamente se desarrolló el sistema de guardado y carga de mapas, que permite almacenar el estado de cada representación en disco y recuperarlo en sesiones posteriores sin necesidad de conectarse al robot. Esta funcionalidad resultó especialmente útil durante las fases de desarrollo y prueba, al permitir trabajar con datos reales sin depender de la disponibilidad del robot o del simulador.

Posteriormente se implementó la integración de la realidad virtual en Unity, haciendo uso del paquete XR Interaction Toolkit para añadir el jugador y sus controles. A continuación se implementó el sistema de puntos de vista, que permite al operador alternar entre distintas perspectivas dentro del entorno virtual, incluyendo vistas en primera y tercera persona del robot virtual, que además permite su teleoperación, y la propia primera persona del jugador.

El minijuego de exploración y alcance de objetivos fue desarrollado en una fase posterior, una vez que el resto del sistema era funcional y estable. Se diseñaron cuatro simples niveles con mecánicas progresivas que introducen la colaboración entre el operador y el robot, apoyándose en el mapa de ocupación para determinar posiciones libres aleatorias en las que generar los objetivos. La lógica del minijuego se validó primero en el simulador y posteriormente con el robot real.

A lo largo de todo el desarrollo se utilizó GitHub para el control de versiones, permitiendo mantener un historial de cambios y revertir a estados anteriores cuando alguna modificación introducía errores difíciles de depurar. Las reuniones periódicas de seguimiento con el tutor permitieron revisar el avance del proyecto, proponer ajustes y resolver dudas que surgían durante la implementación.

La validación del sistema se realizó principalmente en el entorno simulado con Gazebo, realizando pruebas presenciales con el robot físico en las fases finales del proyecto. Las últimas semanas se dedicaron a la realización de pruebas, la corrección de pequeños errores y la redacción de esta memoria.

En cuanto a la distribución temporal, el proyecto se desarrolló entre enero de 2025 y junio de 2026, con una dedicación mayor en las fases de implementación de los sistemas de visualización y en la integración final de todos los componentes.

1.4.1. Cronograma



1.4.2. Explicación por tarea

- **Investigación inicial y estudio de tecnologías:** Se analizaron los distintos enfoques disponibles para la representación tridimensional en Unity, las opciones de integración con ROS para la recepción de datos de senso-

res, y el funcionamiento del sistema de transformaciones TF necesario para proyectar correctamente los datos en el espacio de Unity.

- **Implementación de los sistemas de visualización 3D:** Se desarrollaron de forma independiente los cuatro modos de representación del mapa tridimensional. Cada sistema fue validado por separado antes de integrarse bajo el gestor de mapas común que permite alternar entre ellos en tiempo de ejecución.
- **Sistema de guardado y carga de mapas:** Se implementó la funcionalidad de persistencia de datos para cada tipo de mapa, permitiendo guardar el estado de la representación en disco y recuperarlo posteriormente sin necesidad de conexión al robot.
- **Sistema de puntos de vista:** Se implementó el conjunto de realidad virtual para el jugador, utilizando el XR Interaction Toolkit. Además se desarrolló el sistema de gestión de perspectivas, permitiendo al operador alternar entre las distintas cámaras disponibles mediante los controles del visor de realidad virtual.
- **Desarrollo del minijuego:** Se diseñaron e implementaron los cuatro niveles del minijuego, incluyendo la lógica de generación de objetivos en posiciones libres del mapa, la detección de condiciones de éxito y la gestión del estado de juego entre niveles.
- **Integración y pruebas en simulador:** Se realizó la integración conjunta de todos los módulos y se llevaron a cabo pruebas funcionales en el entorno simulado con Gazebo, verificando el comportamiento correcto del sistema completo.
- **Pruebas con el robot real:** Se validó el funcionamiento del sistema sobre el Turtlebot2 físico, detectando y corrigiendo diferencias de comportamiento respecto al entorno simulado.
- **Redacción de la memoria:** Se elaboró la documentación del proyecto, describiendo las decisiones de diseño, la implementación de cada componente y los resultados obtenidos durante las pruebas.

2

Estado del arte

En este capítulo se analiza el conjunto de trabajos, tecnologías y herramientas previos que constituyen el punto de partida del proyecto. Se estudian las principales formas de integración entre ROS y Unity, las técnicas de representación tridimensional de entornos robóticos, las técnicas de optimización de renderizado en tiempo real para realidad virtual, los sistemas de persistencia de mapas, la reconstrucción de espacios reales como gemelos digitales y el uso de entornos tridimensionales reales en videojuegos y experiencias interactivas.

2.1. Integración ROS-Unity para visualización 3D

La integración entre ROS y Unity para visualización tridimensional en tiempo real ha sido objeto de múltiples trabajos académicos y constituye la base sobre la que se construyen el resto de componentes de este proyecto.

El trabajo de Codd-Downey et al. [5] fue uno de los primeros en proponer un *middleware* basado en *sockets* para intercambiar datos de sensores entre ambos entornos, antes de que existieran herramientas consolidadas como ROS#. Aunque su implementación está ya obsoleta, establece los principios arquitectónicos que siguen siendo relevantes, como la separación entre el nodo de comunicación y la lógica de visualización, y uso de un hilo dedicado para la recepción de datos de red para no bloquear el bucle de renderizado.

Whitney et al. [21] desarrollaron ROS Reality, un *framework* completo para teleoperación y manipulación robótica mediante realidad virtual con *hardware* de consumo. Su sistema integra visualización tridimensional del robot, datos de sensores en tiempo real y control mediante mandos VR, todo sobre ROS y Unity. La arquitectura propuesta utilizando suscriptores ROS que alimentan buffers consumidos por el bucle de renderizado de Unity, es conceptualmente la misma que siguen los componentes desarrollados en este proyecto. Su trabajo demuestra además que la latencia introducida por **rosbridge** sobre *WebSocket* es suficientemente baja para aplicaciones interactivas en tiempo real, aunque puede convertirse en un cuello de botella para flujos de datos pesados.

Hernández et al. [8] presentaron más recientemente un sistema de teleoperación basado en Unity y ROS con **rosbridge**, publicando comandos de movimiento desde Unity y visualizando la cámara del robot en el entorno virtual. Sus resultados confirman que, para aplicaciones que no requieren de alta precisión, **rosbridge** y ROS# constituyen soluciones robustas y eficientes, respaldando la elección del uso de estas tecnologías en este proyecto.

Un aspecto crítico en cualquier sistema de integración entre ROS y Unity es la gestión del árbol de transformaciones TF de ROS. El sistema TF mantiene un grafo de relaciones espaciales entre los distintos marcos de referencia del robot, como el del sensor, el de la base móvil o el marco del mapa, actualizándolas continuamente a medida que el robot se mueve. Utilizar datos de sensores como nubes de puntos en Unity sin tener en cuenta este árbol de transformaciones produce visualizaciones incorrectas, ya que los puntos se proyectan en el espacio sin tener en cuenta la postura real del sensor en el momento de la captura. Implementar correctamente un gestor de transformaciones en Unity que recoja y sincronice los datos del sensor con la posición del robot es un requisito técnico muy importante, abordado en múltiples trabajos académicos [21, 8].

La alternativa más moderna a ROS# es Unity Robotics Hub [15], el paquete oficial de Unity Technologies con soporte nativo para ROS 2, importación de modelos URDF y comunicación basada en TCP en lugar de *WebSockets*. Sin embargo, su compatibilidad con ROS 1 y en particular con ROS Kinetic es muy limitada, lo que lo hace inviable para el Turtlebot2. Esta situación ilustra una problemática habitual en robótica académica entre la disponibilidad de plataformas *hardware legacy* y la evolución del ecosistema *software*.

2.2. Representación tridimensional del entorno en robótica

La capacidad de representar el entorno físico de un robot de forma tridimensional en tiempo real es una de las áreas más activas en robótica móvil. Mientras

que los mapas de ocupación 2D, como los generados por gmapping, ofrecen una representación compacta y suficiente para navegación plana, se quedan cortos en aplicaciones más complejas como la teleoperación o para la exploración y representación fiel del entorno, ya que para esto es necesario conocer la tridimensionalidad completa del espacio.

El formato estándar para la transmisión de nubes de puntos en ROS es el mensaje `sensor_msgs/PointCloud2`, que codifica una nube de puntos arbitraria como un array binario con metadatos que describen los campos disponibles (coordenadas x, y, z, color RGB, intensidad, etc.), la densidad de la nube y el *frame* de referencia al que pertenece. Este formato es producido directamente por los drivers de cámaras RGB-D como la Orbbec Astra del Turtlebot2. Deserializar estos datos en Unity requiere de interpretar un array binario de acuerdo a los metadatos del mensaje, extrayendo los campos de interés y proyectándolos en el espacio virtual utilizando la transformación TF adecuada.

Las cámaras RGB-D, como la Orbbec Astra del Turtlebot2 o la familia Kinect de Microsoft, popularizaron el uso de nubes de puntos densas para la reconstrucción de entornos interiores. Henry et al. [7] demostraron que es posible construir modelos densos y precisos de entornos interiores mediante la acumulación de capturas RGB-D con seguimiento de cámara, obteniendo representaciones con error medio inferior a 20 cm en espacios de varios de metros cuadrados.

Una alternativa más eficiente en memoria a las nubes de puntos densas es la representación octree, formalizada en el trabajo de Hornung et al. [9] con la librería *OctoMap*. En lugar de almacenar puntos individuales, *OctoMap* discretiza el espacio en una jerarquía de cubos de tamaño variable y almacena la probabilidad de ocupación de cada cubo. Esto permite representar entornos complejos con millones de puntos con un consumo de memoria muy inferior al de una nube de puntos equivalente, a costa de perder la información de color del sensor. *OctoMap* se integra de forma nativa con ROS a través del paquete `octomap_server`, que publica el mapa como un mensaje `sensor_msgs/PointCloud2` con los centros de los vóxeles ocupados, lo que facilita su consumo desde aplicaciones externas como Unity sin necesidad de deserializar el formato binario propio de *OctoMap*.

En cuanto al streaming de nubes de puntos para telepresencia, Barone et al. [2] abordan el problema de la transmisión eficiente de datos de nube de puntos en tiempo real para teleoperación inmersiva, que es la referencia más directamente relevante para las decisiones de diseño del nodo `pointcloud_relay.py` de este proyecto. Su trabajo demuestra que la decimación espacial, es decir, conservar únicamente uno de cada *n* puntos, combinada con la limitación de la frecuencia de publicación, es una estrategia efectiva para reducir el ancho de banda necesario sin degradar de forma perceptible la calidad de la representación desde el punto de vista del usuario. Sus experimentos muestran que resoluciones de malla inferiores a la agudeza visual del operador son innecesarias, y que la frecuencia de actualización del mapa tiene un impacto mayor en la sensación de fidelidad

que la densidad de puntos por *frame*.

En el contexto de la telepresencia inmersiva en tiempo real, Reality Fusion [10] son representativos en fusión volumétrica. Los autores proponen una arquitectura que combina datos RGB, profundidad y odometría para reconstruir el entorno tridimensional del robot, actualizada desde una interfaz de VR. El sistema alcanza representaciones densas a varios fotogramas por segundo sobre *hardware* de consumo, aunque su complejidad computacional supera considerablemente los recursos disponibles en una plataforma académica con Turtlebot2.

2.3. Técnicas de renderizado eficiente en tiempo real

La representación de grandes volúmenes de datos geométricos en tiempo real con tasas de refresco suficientes para realidad virtual es uno de los problemas técnicos centrales de este proyecto, ya que se el mapa se construye en tiempo real, por lo que es necesario mantener el rendimiento no solo con el mapa ya construido si no también, mientras se genera. Unity dispone de varias herramientas para abordar este problema, cuyo uso adecuado determina en gran medida si el sistema puede mantener los 90 FPS recomendados para experiencias VR cómodas.

El GPU Instancing es la técnica más relevante para la representación de grandes cantidades de objetos idénticos, como los vóxeles de un mapa 3D. Mediante la función de Unity para instanciar mallas, se puede renderizar hasta 1023 instancias de un mismo mesh en una única llamada de renderizado, reduciendo drásticamente la carga sobre la CPU respecto a tener un GameObject por vóxel. Para mapas con decenas de miles de vóxeles, la diferencia entre ambos enfoques puede ser determinante en la tasa de fotogramas. La limitación de 1023 instancias por llamada se gestiona dividiendo el mapa en lotes y realizando múltiples llamadas [17, 1].

Por otro lado, el sistema de partículas de Unity, es otra herramienta optimizada para renderizar grandes cantidades de elementos simples. Unity se encarga de gestionar internamente las partículas en un buffer en memoria y las envía a la GPU en un único lote, independientemente del número de partículas activas. La API de partículas permite controlar completamente sus características como la posición, color y tamaño de cada partícula sin simulación física, lo que la convierte en una alternativa eficiente frente a los GameObjects individuales [19, 1].

La generación de mallas procedurales es la técnica más flexible pero también la más costosa. Construir una malla con decenas de miles de vértices en el hilo principal de Unity puede causar picos de latencia en la CPU. El uso de corrutinas para distribuir la construcción de la malla en múltiples *frames* es la práctica

recomendada por la documentación oficial de Unity para operaciones de generación procedimental de geometría, ya que permite repartir la carga computacional entre varios ciclos del bucle de juego sin bloquear el renderizado [18, 6, 16].

El *pool* de objetos es una técnica fundamental en desarrollo de videojuegos para evitar la presión sobre el recolector de basura de .NET, que en Unity puede introducir caídas de fotogramas cuando libera grandes cantidades de objetos. Nystrom [12, 11] documenta este patrón como uno de los fundamentales en programación de videojuegos, mantener un conjunto de objetos preinstanciados que se activan y desactivan en lugar de instanciarse y destruirse, especialmente para objetos que se crean y eliminan con frecuencia.

2.4. Sistemas de persistencia de mapas robóticos

La persistencia de mapas robóticos es un requisito habitual en aplicaciones de robótica de servicio, donde el robot necesita recordar el entorno entre sesiones sin repetir el proceso de exploración. En el ecosistema ROS, el paquete `map_server` proporciona herramientas para guardar y cargar mapas de ocupación 2D en formato PGM con metadatos YAML, pero no existe un equivalente estándar para mapas 3D densos.

OctoMap [9] incluye su propio formato binario de serialización (`.ot` y `.bt`), compacto y eficiente pero específico de esa librería. Para nubes de puntos densas con información de color, los formatos estándar en la comunidad de visión por computador son PCD, usado por la Point Cloud Library (PCL), y PLY, que incluye soporte para colores por vértice y es legible por herramientas como CloudCompare o MeshLab.

En el contexto específico de Unity, la serialización a JSON es la opción más accesible para datos estructurados. Para mapas con decenas de miles de puntos el tamaño del archivo puede superar varios megabytes, lo que hace que la lectura y escritura sean más lentas que con formatos binarios. A pesar de esto, la legibilidad del JSON facilita la depuración y la interoperabilidad con otras herramientas, justificando su uso en un contexto académico, valorando la simplicidad.

2.5. Gemelos digitales y reconstrucción de espacios reales

Un gemelo digital es una representación virtual de un objeto, espacio o sistema físico que se construye y actualiza a partir del mundo real. En el ámbito de

los espacios interiores, la creación de gemelos digitales precisos ha cobrado gran relevancia en los últimos años gracias al abaratamiento de los sensores de profundidad y los escáneres láser, y a la disponibilidad de plataformas de procesamiento accesibles.

Los escáneres LiDAR terrestres de alta precisión constituyen la solución estándar en entornos profesionales como arquitectura o patrimonio cultural [20]. Estos dispositivos capturan nubes de puntos de millones de puntos con precisión milimétrica en pocos minutos. Sin embargo, su coste y complejidad operativa los hacen inaccesibles para plataformas académicas. Una alternativa de bajo coste es el uso de cámaras RGB-D montadas en robots móviles, que permiten acumular capturas sucesivas mientras el robot recorre el espacio. Trabajos como el de Henry et al. [7] demostraron la viabilidad de esta aproximación para reconstruir entornos interiores con errores menores, mediante el registro iterativo de nubes de puntos.

En el ecosistema ROS, *OctoMap* [9] es la solución más utilizada para construir representaciones volumétricas tridimensionales del entorno en tiempo real a partir de los datos de un sensor LiDAR o RGB-D. Su estructura de datos en un árbol octal permite actualizar el mapa de forma incremental a medida que el robot explora nuevas zonas, y su integración nativa con ROS facilita su uso en plataformas como el Turtlebot2. El resultado es un mapa de vóxeles que puede ser utilizado en aplicaciones externas, constituyendo la base del gemelo digital del espacio real en este proyecto.

La fotogrametría es otra técnica ampliamente utilizada para la generación de modelos 3D de espacios reales a partir de fotografías 2D convencionales. Herramientas como Meshroom reconstruyen mallas tridimensionales texturizadas mediante algoritmos de Structure from Motion y Multi-View Stereo. Aunque producen modelos de mayor fidelidad visual que las nubes de puntos LiDAR o RGB-D, requieren un proceso de captura más delicado, además de un tiempo de procesamiento en diferido considerable, lo que las hace menos adecuadas para reconstrucción en tiempo real desde un robot en movimiento [13].

El uso de gemelos digitales en entornos industriales está ampliamente documentado. Boje et al. [4] revisan la integración de gemelos digitales con Building Information Modeling (BIM) para la gestión de edificios, mostrando que la disponibilidad de una representación 3D precisa y actualizada del espacio físico permite automatizar tareas de supervisión, mantenimiento y planificación. En un contexto académico y robótico, el mismo principio se aplica en una menor escala, ya que disponer de un gemelo digital del espacio en el que opera el robot permite razonar sobre él, interactuar con él y usarlo como base para aplicaciones que van más allá del simple mapeo.

2.6. Entornos 3D reales en videojuegos y experiencias interactivas

La integración de datos capturados del mundo real en experiencias interactivas y videojuegos es una tendencia consolidada en la industria del entretenimiento y en la investigación en interacción entre humanos y ordenadores. La posibilidad de explorar, habitar y jugar en una representación digital de un espacio físico real añade una dimensión de autenticidad que los entornos puramente artificiales no pueden ofrecer.

En el ámbito de los videojuegos, la fotogrametría se ha convertido en una técnica estándar para capturar elementos del mundo real con alta fidelidad visual. Títulos como *Star Wars Battlefront* (DICE, 2015) o *The Vanishing of Ethan Carter* (The Astronauts, 2014) utilizaron fotogrametría de objetos y espacios reales para construir sus entornos de juego, obteniendo una calidad visual difícil de alcanzar mediante modelado manual [22].

Las nubes de puntos como representación interactiva han encontrado su espacio principalmente en experiencias de realidad virtual. Trabajos como el de Schütz et al. [14] desarrollaron sistemas de renderizado en tiempo real de nubes de puntos masivas para aplicaciones web y de visualización, demostrando que es posible navegar interactivamente por capturas de espacios reales con millones de puntos sobre *hardware* de consumo. En el contexto de la realidad virtual, esta capacidad permite al usuario explorar espacios reales digitalizados con sensación de presencia.

La exploración de espacios reales escaneados en VR ha sido estudiada en aplicaciones de patrimonio cultural y arquitectura. Bekele et al. [3] revisaron el uso de realidad virtual para la exploración de sitios digitalizados y concluyeron que la correspondencia entre el espacio virtual y el espacio físico original aumenta significativamente la sensación de presencia y el valor educativo de la experiencia, incluso cuando la fidelidad geométrica es moderada. Este resultado es relevante para este proyecto, ya que el mapa generado por el Turtlebot no pretende competir en fidelidad con un escáner de precisión profesional, pero sí mantener la correspondencia espacial con el entorno real.

En cuanto a la combinación de exploración de entornos reales con mecánicas de juego, los llamados juegos serios han explorado esta unión en contextos educativos y de formación. Zyda [25] define los juegos serios como aplicaciones que combinan entretenimiento con el aprendizaje o entrenamiento, y señala que el uso de entornos correspondientes con el mundo real aumentan la transferencia de lo aprendido al contexto real. En robótica, el uso de entornos simulados con correspondencia con el mundo real para entrenamiento e interacción ha sido destacado recientemente [24], ya que permite diseñar tareas estructuradas cuya dificultad puede regularse de forma controlada.

La elección del punto de vista dentro de un entorno 3D interactivo influye directamente en la experiencia del usuario. En videojuegos, la primera persona ofrece la mayor inmersión pero reduce la conciencia espacial global, mientras que la tercera persona facilita esta comprensión del entorno [23]. Para experiencias de exploración de espacios reales en VR, la posibilidad de alternar entre perspectivas según la tarea se valora positivamente por parte de los usuarios, ya que cada perspectiva revela información distinta del mismo espacio tridimensional [3].

2.7. Comparativa y posicionamiento del proyecto

Los trabajos revisados en este capítulo cubren de forma parcial los distintos componentes del sistema desarrollado, pero ninguno los integra de forma conjunta con el enfoque específico de este proyecto. Se pueden destacar las siguientes características diferenciadoras:

1. Los sistemas de visualización 3D revisados, como Reality Fusion [10] o los trabajos de Whitney et al. [21], implementan una única representación del entorno. Este proyecto ofrece cuatro representaciones seleccionables en tiempo de ejecución, cada una con diferente compromiso entre fidelidad, rendimiento y coste computacional.
2. El trabajo de Barone et al. [2] aborda la retransmisión eficiente de nubes de puntos pero no incluye representación en Unity ni una experiencia interactiva. Este proyecto cierra ese ciclo, integrando decimación, recepción, procesamiento, visualización y uso interactivo del mapa en un único sistema.
3. Los trabajos sobre exploración de espacios reales en VR revisados, como Bekele et al. [3], utilizan modelos obtenidos mediante fotogrametría o escáneres de precisión, procesados *offline*. Este proyecto construye el gemelo digital en tiempo real desde un robot de bajo coste, sin procesamiento *offline*.
4. Ninguno de los trabajos revisados integra un sistema de persistencia que permita guardar el mapa capturado y reutilizarlo en sesiones posteriores sin conectar el robot, ni un minijuego de exploración jugable, con el jugador y el robot en distintas perspectivas, tanto sobre el mapa capturado en tiempo real como sobre uno importado previamente.

La aportación principal del proyecto es la integración funcional de un *pipeline* completo que va desde la captura de datos de sensor en un robot real hasta una experiencia interactiva en realidad virtual, sobre *hardware* de bajo coste y *software* de acceso libre.

3

Diseño

En este capítulo se describe el diseño general del proyecto, la integración de sus sistemas y el presupuesto requerido.

3.1. Diseño del sistema

El diseño del sistema se planteó con el objetivo de permitir una experiencia de teleoperación inmersiva en realidad virtual, donde el usuario pudiera controlar un robot móvil y percibir su entorno de forma natural y con baja latencia. La arquitectura debía integrar dispositivos heterogéneos: visores de realidad virtual, ordenador de ejecución y plataforma robótica, manteniendo una experiencia interactiva fluida y consistente desde el punto de vista del usuario.

Para ilustrar esta estructura global, la Figura 3.1 muestra el diagrama de arquitectura del sistema. Este diagrama cambia al usar el entorno simulado. En este caso, el robot coexiste con un espacio virtual dentro de una máquina virtual Ubuntu 16 mediante Gazebo.

A nivel conceptual, el sistema se organiza como una arquitectura modular compuesta por tres bloques principales: el entorno inmersivo de visualización y control, el nodo de integración y procesamiento, y la plataforma robótica. Cada bloque asume responsabilidades claramente separadas, reduciendo el acoplamiento entre componentes y facilitando tanto el mantenimiento como la evolución futura del sistema.

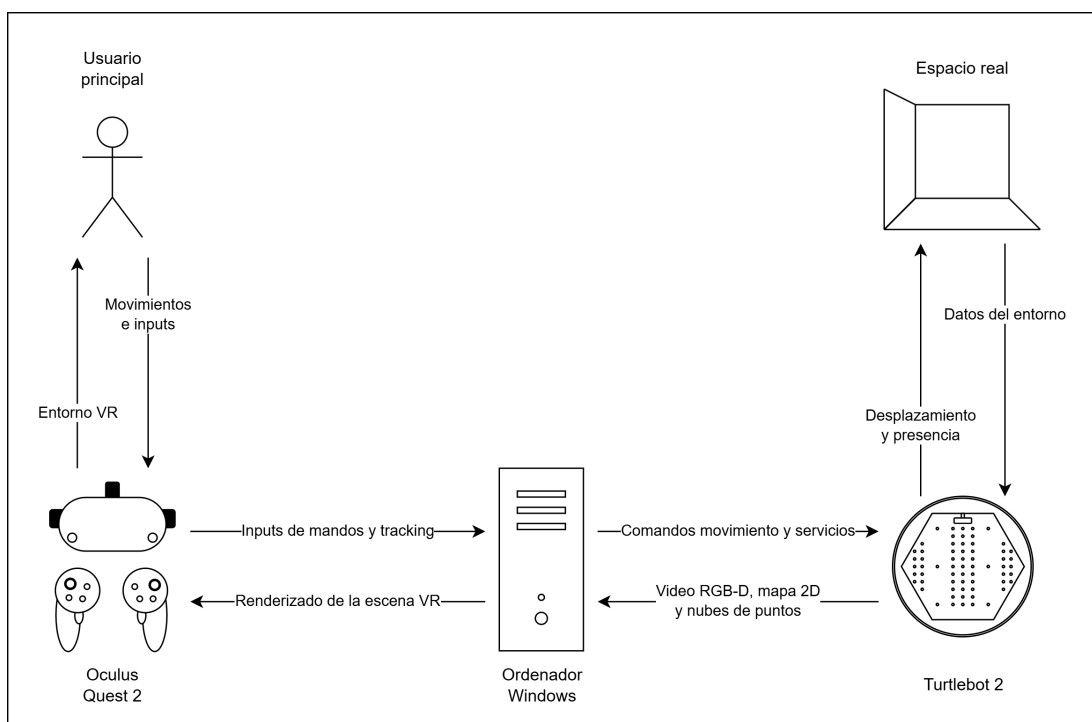


Figura 3.1: Diagrama de la arquitectura del sistema.

3.2. Criterios de diseño

La arquitectura se definió a partir de tres criterios principales:

- **Inmersión y continuidad visual:** La experiencia en realidad virtual exige mantener una actualización visual estable y una latencia reducida, evitando interrupciones que degraden la sensación de presencia.
- **Separación entre renderizado y comunicaciones:** Las tareas relacionadas con la reconstrucción del entorno virtual no debían verse bloqueadas por retrasos en la recepción de datos externos.
- **Escalabilidad del entorno interactivo:** El sistema debía permitir añadir nuevas fuentes de información o nuevas herramientas de interacción sin rediseñar la lógica principal de la aplicación.

3.3. Decisiones arquitectónicas

A partir de estos criterios se adoptaron tres decisiones principales.

3.3.1. Topología centralizada

El ordenador central actúa como núcleo de procesamiento del sistema. En este bloque convergen todos los flujos de información procedentes del robot, y es aquí donde se realiza la reconstrucción del entorno tridimensional y el renderizado de la escena inmersiva. Además, este nodo traduce las acciones del usuario en comandos de navegación y coordina la sincronización entre percepción, simulación e interacción. Centralizar esta responsabilidad simplifica el diseño y evita dependencias directas entre el visor y la plataforma robótica.

3.3.2. Separación de renderizado y comunicaciones

El sistema distingue conceptualmente entre el entorno inmersivo de visualización y el procesamiento de red. Desde el punto de vista del diseño, el visor de realidad virtual se trata como una capa de interacción desacoplada de la lógica de sensores. Esta separación responde a que ambos presentan requisitos distintos; mantener el renderizado independiente de los datos externos evita compromisos de diseño innecesarios y mejora la fluidez global del sistema.

3.3.3. Desacoplamiento de las plataformas físicas

El visor de realidad virtual y el robot físico se gestionan como nodos independientes. El robot es un proveedor de percepción y un actuador de navegación, abstraído de la lógica visual del sistema. Por su parte, el visor tiene la doble función de capturar interacciones y presentar el entorno. Esta decisión facilita añadir nuevas herramientas sin modificar la arquitectura base.

3.4. Flujo de interacción

El sistema se organiza alrededor de dos ciclos principales de información:

1. **Ciclo de percepción:** El robot captura continuamente información del entorno y la envía al nodo central, donde se transforma en una representación virtual interpretable por el usuario.
2. **Ciclo de control:** El usuario interactúa con el entorno inmersivo mediante movimientos e inputs de los mandos. Estas acciones se traducen en comandos de navegación que se envían al robot.

Esta separación permite que percepción y control evolucionen de forma parcialmente independiente, mejorando la capacidad de respuesta del sistema y manteniendo la fluidez necesaria para una experiencia inmersiva satisfactoria.

3.5. Presupuesto y costes

En esta sección se detalla el análisis económico del proyecto, diferenciando entre los costes de material y los de personal. Las 300 horas de dedicación corresponden a la carga teórica de un Trabajo de Fin de Grado de 12 créditos ECTS. El tiempo adicional invertido debido a la curva de aprendizaje y a los problemas de compatibilidad con el *software legacy* se considera formación y se excluye del cómputo para no distorsionar el presupuesto. Todo el *hardware* utilizado era pre-existente, bien de propiedad personal o cedido por el laboratorio, y el *software* empleado es en su totalidad de acceso libre o con licencia académica gratuita. Por tanto, el coste directo de adquisición ha sido por tanto nulo.

A continuación se recoge el valor comercial estimado de los recursos utilizados, con el fin de reflejar el coste real que tendría replicar el proyecto desde cero.

Disponer del PC de sobremesa en el mismo espacio que el robot habría eliminado la necesidad del portátil para las pruebas presenciales. Además, el dispositivo móvil solo se utilizó en las pruebas presenciales como punto de acceso WiFi, para lo cual una alternativa mucho más económica hubiera sido disponer de un router WiFi.

El coste total de replicación del proyecto se reduce al valor de la mano de obra, ya que todos los recursos materiales estaban disponibles sin coste adicional. De haber sido necesario adquirir el *hardware* y cubrir el coste de personal, el desembolso total habría ascendido a aproximadamente 8.307,55 €.

Recurso material	Uso	Valor Comercial	Coste Real
<i>Hardware de Laboratorio y Pruebas</i>			
Turtlebot 2	Plataforma robótica para captura del entorno	1.400,00 €	0,00 €
Meta Quest 2	Visor VR para la experiencia interactiva	349,00 €	0,00 €
Portátil para pruebas presenciales	Validación del sistema con el robot real	1.000,00 €	0,00 €
<i>Hardware Personal</i>			
PC de sobremesa para desarrollo	Programación, simulación y desarrollo Unity	1.258,56 €	0,00 €
Smartphone Android	Punto de acceso WiFi durante las pruebas	249,99 €	0,00 €
<i>Software y Licencias</i>			
Unity 2022	Motor gráfico (Licencia Personal/Student)	0,00 €	0,00 €
ROS Kinetic + Ubuntu 16.04	Entorno del robot (Código abierto)	0,00 €	0,00 €
Librerías y paquetes externos	ROS#, OctoMap, XR Toolkit, etc.	0,00 €	0,00 €
Total Recursos Materiales		4.257,55 €	0,00 €

Tabla 3.1: Desglose de recursos materiales, valor comercial y coste real.

Fase del Proyecto	Horas Asignadas	Precio / Hora	Coste Total
Diseño e implementación del sistema	200 h	13,50 €	2.700,00 €
Pruebas, despliegue presencial y depuración	70 h	13,50 €	945,00 €
Redacción de documentación y memoria	30 h	13,50 €	405,00 €
Total Mano de Obra	300 h	-	4.050,00 €

Tabla 3.2: Desglose de los costes de personal.

4

Desarrollo

En este capítulo se describe el proceso de implementación del sistema, detallando las decisiones de diseño tomadas, las tecnologías empleadas y el funcionamiento de cada componente desarrollado.

4.1. Plataforma de desarrollo

A continuación se describen los elementos de *hardware* y *software* empleados durante el desarrollo de este proyecto.

4.1.1. Hardware

- El **TurtleBot 2** se utilizó como plataforma robótica móvil para el escaneo del entorno. El robot incorpora una cámara RGB-D Orbbec Astra para la captura de imágenes y datos de profundidad, así como una base móvil Kobuki que permite su desplazamiento.
- Las **Meta Quest 2** se utilizaron como dispositivo de visualización e interacción en realidad virtual, proporcionando una representación estereoscópica del entorno que permite al usuario estar presente en el mapa 3D generado.
- Como equipo principal de desarrollo se utilizó un **ordenador de sobremesa** con sistema operativo Windows 11 Pro, equipado con una tarjeta gráfica Nvidia RTX 5060 Ti con 16 GB de memoria VRAM, un procesador AMD

Ryzen 7 7700, 32 GB de memoria RAM DDR5 y 1 TB de almacenamiento. Este equipo permitió ejecutar simultáneamente el proyecto de Unity y una máquina virtual con Ubuntu 16.04 para la simulación del Turtlebot en Gazebo, además de otras herramientas auxiliares.

- Para las pruebas con el robot físico se utilizaron distintos **ordenadores portátiles** disponibles en el laboratorio, con capacidad suficiente para ejecutar la aplicación de Unity y comunicarse con el robot.
- También se utilizó un **teléfono móvil** como punto de acceso Wi-Fi durante las pruebas presenciales, para establecer una red local entre los dispositivos.

4.1.2. Software

- **ROS Kinetic** se utilizó como *middleware* para gestionar la comunicación entre los componentes del sistema robótico, tanto en la simulación con Gazebo 7 como en la integración con el Turtlebot2 físico.
- **ROS#** se utilizó como capa de comunicación entre Unity y ROS, permitiendo suscribirse a topics y publicar mensajes desde C# de forma transparente. Utiliza la tecnología de **rosbridge** sobre *WebSocket*.
- **Unity 2022** fue el motor de desarrollo para la aplicación de realidad virtual. A través de él se implementaron la visualización del mapa, la interacción del usuario y las mecánicas del minijuego. Para la integración con las Meta Quest 2 se emplearon los paquetes XR Interaction Toolkit y XR Plugin Management.
- **Gazebo 7** se utilizó como entorno de simulación durante la mayor parte del desarrollo, permitiendo validar las distintas funcionalidades sin necesidad de acceder al robot físico.
- **Ubuntu 16.04** fue el sistema operativo sobre el que corren ROS Kinetic y Gazebo 7, ejecutado como máquina virtual mediante Hyper-V y Oracle VirtualBox según las necesidades de cada fase del desarrollo.
- Se utilizaron **Hyper-V** y **Oracle VirtualBox** como plataformas de virtualización para ejecutar Ubuntu 16.04 sobre el sistema anfitrión Windows 11 Pro.

4.1.3. Herramientas adicionales

- **Visual Studio 2022** se utilizó como entorno de desarrollo integrado para la implementación y depuración del código de Unity en C#.

- **GitHub** se utilizó para el control de versiones, permitiendo mantener un historial de cambios y revertir a estados anteriores cuando alguna modificación introducía errores difíciles de depurar.

4.2. Arquitectura general del sistema

El sistema se organiza en cuatro bloques que trabajan de forma coordinada. El primero es el robot Turtlebot2, que ejecuta el *stack* de ROS con los nodos de navegación, SLAM y la cámara RGB-D. Es la fuente de todos los datos del entorno real. El segundo es la aplicación de Unity, que actúa como núcleo del sistema. Recibe los datos del robot, los procesa y los representa en el entorno virtual. También gestiona la interacción del usuario y la lógica del minijuego. El tercero es la conexión entre ambos, que se realiza mediante **rosbridge** sobre *WebSocket* usando el paquete ROS#. Por este canal viajan el mapa de ocupación, los datos de odometría, las transformadas TF, las nubes de puntos decimadas y los comandos de movimiento. El cuarto es el propio usuario, que interactúa con el entorno virtual a través de las gafas Meta Quest 2 y sus mandos, tanto para explorar el mapa como para participar en el minijuego junto al robot.

En cuanto a las decisiones de diseño que afectan a todo el sistema, la más importante es la separación entre la recepción de datos y su uso. Los callbacks de ROS llegan en hilos secundarios, de forma que los datos se almacenan en estructuras internas y se consumen desde el hilo principal de Unity en cada *frame*. Esto evita condiciones de carrera y mantiene la escena estable. Otra decisión transversal es el límite de vóxeles configurado en todos los mapas, que evita que la memoria crezca indefinidamente durante sesiones *largas* de exploración.

4.3. Entorno ROS en el Turtlebot2

4.3.1. Stack de software en el robot

El Turtlebot2 ejecuta Ubuntu 16.04 con ROS Kinetic. Para disponer de los datos necesarios para la reconstrucción tridimensional del entorno, se lanzan secuencialmente los siguientes procesos:

1. `roslaunch turtlebot_bringup minimal.launch` inicializa los *drivers* de *hardware* de Kobuki: motores, sensores de choque y de acantilado, batería y odometría.
2. `roslaunch turtlebot_navigation gmapping_demo.launch` lanza el *stack* de navegación con SLAM basado en gmapping, que construye el mapa de

ocupación 2D a partir del láser y lo publica en `/map`. Este mapa es la fuente de datos del *Wall Map*.

3. `roslaunch rosbridge_server rosbridge_websocket.launch` expone los topics y servicios de ROS como una API *WebSocket* en el puerto 9090, canal por el que Unity recibe todos los datos descritos en este capítulo.
4. `pointcloud_relay.py` nodo personalizado que decima y limita la frecuencia de la nube de puntos de la cámara RGB-D antes de publicarla, para que pueda transmitirse por `rosbridge` sin saturar el *WebSocket*. Alimenta el *Mesh Map* y el *Particle Map*.
5. `roslaunch octomap_server octomap_color_server_node` `cloud_in:=/camera/depth_registered/points` `resolution:=0.05` `frame_id:=map` construye el mapa volumétrico de vóxeles a partir de la nube de puntos completa, aplicando fusión y filtrado de ruido. Alimenta el *Octo Map*.

En el simulador, los dos primeros comandos se sustituyen por `roslaunch turtlebot_gazebo turtlebot_world.launch` y `roslaunch turtlebot_gazebo gmapping_demo.launch`; el resto del *stack* se mantiene igual.

4.3.2. Nodo de decimación de nube de puntos `pointcloud_relay.py`

La cámara RGB-D del Turtlebot2 publica en `/camera/depth_registered/points` nubes de entre 300.000 y 600.000 puntos por *frame*. Transmitir ese volumen directamente por `rosbridge` saturaría el *WebSocket* y bloquearía el resto de la comunicación con Unity, por lo que este nodo actúa como filtro intermedio antes de la publicación.

El nodo se suscribe a la nube original y repubblica una versión reducida en `/camera/depth_registered/points_decimated`. La reducción combina dos estrategias: decimación espacial, conservando uno de cada diez puntos mediante slicing sobre la lista de puntos leída con `point_cloud2.read_points`, y limitación temporal, descartando mensajes si no ha transcurrido al menos 0,5 segundos desde la última publicación (máximo 2 Hz). La nube decimada se reconstruye con `point_cloud2.create_cloud`, conservando los campos de posición y color RGB.

Esta doble estrategia reduce el volumen de datos en más de un orden de magnitud, garantizando que el *WebSocket* reciba un flujo manejable sin perder la estructura general del entorno capturado.

4.3.3. Nodo de mapa volumétrico `octomap_server`

A diferencia de los demás nodos descritos, `octomap_server` no es un script propio sino un paquete estándar de ROS, lanzado con parámetros específicos para este proyecto. Se suscribe directamente a la nube de puntos completa (sin pasar por el relay, ya que el procesamiento ocurre en el propio robot) y construye un mapa de ocupación 3D mediante una estructura octree, fusionando capturas sucesivas y filtrando ruido del sensor.

Los parámetros empleados son una resolución de vóxel de 0,05 m, equivalente a la usada por defecto en el resto de mapas del proyecto, y el *frame* de referencia *map*, para que el mapa resultante ya esté expresado en el marco global y no requiera transformación adicional en Unity. El nodo publica los centros de los vóxeles ocupados como un mensaje `sensor_msgs/PointCloud2` en el topic `/octomap_point_cloud_centers`, que es el que consume directamente el componente `OctoMapPointCloudSubscriber` de Unity.

Al ejecutarse sobre la nube completa y no la decimada, este nodo ofrece la representación con menor pérdida de información de los cuatro modos del proyecto, a costa de una frecuencia de actualización menor que el resto.

4.4. Comunicación con ROS y recepción de datos

4.4.1. Conexión Unity y ROS mediante ROS#

La comunicación entre Unity y ROS se establece a través del paquete ROS#, que implementa el protocolo `rosbridge` sobre `WebSocket`. El componente `Ros Connector` gestiona la conexión con el servidor `rosbridge` del robot, y el resto de componentes lo usan para publicar o suscribirse a topics.

Esta arquitectura tiene una ventaja práctica importante, Unity corre en Windows y ROS en Ubuntu, y `rosbridge` hace que eso sea completamente transparente. Cualquier componente de Unity puede comunicarse con ROS sin conocer los detalles de la infraestructura de red.

4.4.2. Suscriptor del mapa de ocupación 2D

El topic `/map` publica mensajes de tipo `nav_msgs/OccupancyGrid`, que representan el mapa construido por gmapping como un array unidimensional de celdas. Cada celda tiene uno de tres valores posibles: 0 si está libre, 100 si está

ocupada y -1 si es desconocida. La celda en la columna x y la fila y se localiza en la posición $y \times \text{ancho} + x$ del array.

El componente almacena estos datos internamente y expone dos operaciones públicas que usan el resto del sistema, convertir una posición del mundo de Unity a coordenadas de celda del mapa, y consultar el estado de ocupación de una celda concreta. La primera opera restando el origen del mapa, dividiendo por la resolución y redondeando hacia abajo. La segunda simplemente indexa el array y devuelve -1 si las coordenadas están fuera de rango. Estas dos operaciones son la base sobre la que el minijuego genera objetivos en zonas libres.

4.4.3. Recepción de nubes de puntos

La cámara RGB-D del robot publica nubes de puntos de cientos de miles de puntos por *frame*, un volumen que saturaría el *WebSocket* de **rosbridge** si se transmitiera sin procesar. Por ello, como se ha descrito en la sección anterior, el robot ejecuta `pointcloud_relay.py` antes de la publicación, dejando un flujo manejable sin perder la estructura esencial del entorno.

En Unity, los receptores deben interpretar el formato binario del mensaje `PointCloud2`, buscando los offsets de los campos de posición y color en los metadatos del propio mensaje. Además, hay que convertir entre sistemas de coordenadas, ya que ROS usa el convenio FLU (X adelante, Y izquierda, Z arriba), mientras que Unity usa X derecha, Y arriba, Z adelante. La conversión es:

$$\begin{pmatrix} x_U \\ y_U \\ z_U \end{pmatrix} = \begin{pmatrix} -y_R \\ z_R \\ x_R \end{pmatrix} \quad (4.1)$$

donde el subíndice R denota coordenadas ROS y U coordenadas Unity.

Para evitar acumular puntos erróneos por el movimiento del robot, existe un componente que hace que los receptores solo acepten datos cuando el robot lleva al menos un segundo quieto.

4.4.4. Detector de movimiento del robot

El componente `RobotMotionDetector` se suscribe al *topic* `/odom` y estima si el robot está quieto calculando la velocidad lineal y angular entre mensajes consecutivos de odometría. La velocidad lineal es la distancia euclídea entre posiciones consecutivas dividida por el intervalo de tiempo; la angular, el ángulo entre orientaciones consecutivas dividido igualmente por el intervalo.

Para evitar que vibraciones menores interrumpen la captura, el robot solo se considera quieto cuando ambas velocidades llevan al menos un segundo por debajo de sus umbrales configurables. Si en algún instante alguna supera su umbral, el contador se reinicia a cero.

4.4.5. Gestión de transformadas temporales

Los puntos de la nube llegan en el sistema de coordenadas de la cámara, pero para acumularlos en un mapa global consistente hay que transformarlos al marco del mapa. ROS gestiona estas relaciones mediante el árbol de transformadas TF, que publica continuamente las posiciones relativas entre todos los marcos de referencia del robot.

El problema es que existe un desfase entre el momento en que se captura la nube y el momento en que llega a Unity. Usar la transformada más reciente en lugar de la del instante de captura haría que los puntos aparecieran ligeramente desplazados, degradando la calidad del mapa.

Para resolverlo, el componente encargado de gestionar las transformadas mantiene un historial de las últimas recibidas por los topics `/tf` y `/tf_static`, con un límite de 1000 entradas para evitar un crecimiento ilimitado de memoria. Cuando se necesita la transformada para un instante concreto, se buscan los dos registros más cercanos mediante búsqueda binaria y se interpola entre ellos: interpolación lineal para traslaciones y SLERP para rotaciones. Si el instante pedido cae fuera del rango del historial, se devuelve la transformada extrema más cercana.

La correcta aplicación de estas transformadas permite que la nube de puntos capturada en ROS se proyecte con exactitud en el espacio de Unity, tal y como evidencia la comparativa cenital de la Figura 4.3.

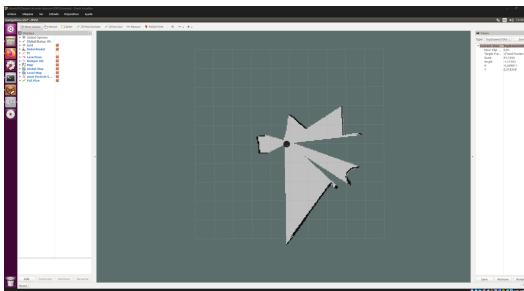


Figura 4.1: *
Visualización en RViz (ROS)

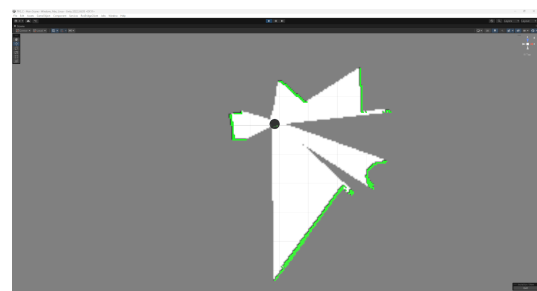


Figura 4.2: *
Proyección en Unity

Figura 4.3: Comparativa espacial cenital que demuestra la correcta sincronización y transformación de coordenadas entre el ecosistema ROS y el motor Unity.

4.5. Representación tridimensional del entorno

El sistema implementa cuatro modos de representación del entorno, todos activos en memoria simultáneamente pero con solo uno visible en cada momento. El usuario puede cambiar entre ellos desde la interfaz sin reiniciar nada. Cada modo tiene un compromiso diferente entre calidad visual, coste computacional y tipo de información que captura.

4.5.1. Wall Map

Es la representación más directa y ligera. Cada celda ocupada del mapa 2D de gmapping se convierte en un cubo vertical en la escena de Unity, formando visualmente las paredes del entorno explorado. Al estar basado en el mapa 2D, solo captura la estructura plana del espacio, sin información de altura ni de objetos pequeños. A cambio, es el modo más estable y eficiente, y el que se muestra por defecto al arrancar.

Para que las actualizaciones frecuentes del mapa no sean costosas, los cubos se reutilizan mediante una *pool* de objetos, en lugar de destruirlos y recrearlos, se desactivan al limpiar el mapa y se recuperan del *pool* cuando hacen falta. Solo se instancian cubos nuevos cuando la *pool* está vacía. Además, todos los cubos comparten un único material, lo que permite dibujarlos en un solo lote y reduce considerablemente el número de llamadas de renderizado.

Además de las paredes, este componente genera una textura 2D a partir de los datos del mapa, con blanco para celdas libres, negro para ocupadas y gris para desconocidas. Esta textura se usa para colocar en el suelo de la escena 3D. El resultado visual de esta extrusión arquitectónica básica se muestra en la Figura 4.4.

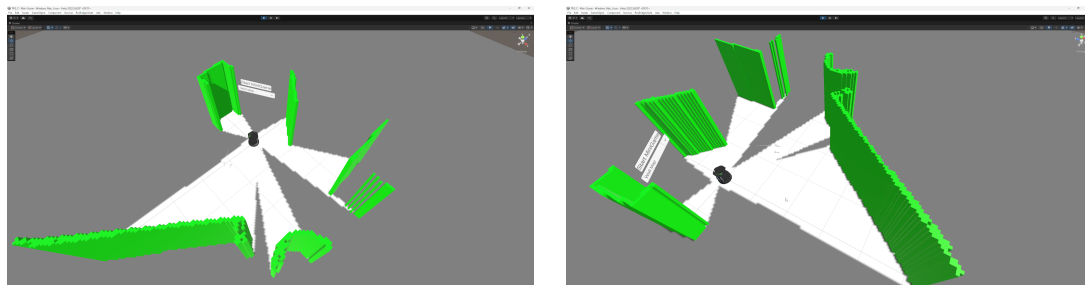


Figura 4.4: Perspectivas del entorno generado mediante la técnica Wall Map, extrayendo la geometría directamente de la cuadrícula de ocupación 2D.

4.5.2. Mesh Map

Este modo acumula los puntos de la nube de puntos como vóxeles y los representa generando una malla de cubos 3D, uno por vóxel. A diferencia del *Wall Map*, captura la geometría completa del entorno, incluyendo suelos, muebles y objetos de cualquier altura. El resultado es una reconstrucción sólida que se enriquece progresivamente a medida que el robot explora.

Cada vóxel se identifica por sus coordenadas divididas por el tamaño de vóxel, de forma que varios puntos próximos entre sí colapsen en el mismo vóxel, reduciendo la redundancia. El acceso al diccionario de vóxeles desde el hilo de **rosbridge** y desde el hilo principal de Unity se protege mediante un objeto de bloqueo para evitar condiciones de carrera.

El principal reto de este modo es el coste de reconstruir la malla cuando llegan puntos nuevos. Con decenas de miles de vóxeles, generar todos los vértices y triángulos en un único *frame* causa caídas de *framerate* inaceptables en VR. La solución fue distribuir la reconstrucción en varios *frames* mediante una corrutina, en cada *frame* se procesa un bloque de vóxeles y se cede el control al motor, de forma que el renderizado puede continuar entre bloques. Esto mantiene el *framerate* más estable a costa de que la actualización visual tarde un poco más en completarse, lo cual en la práctica no resulta perceptible.

La reconstrucción basada en mallas tridimensionales continuas *Mesh Map* genera superficies más orgánicas y detalladas uniendo los vértices adyacentes. En la Figura 4.7 se muestran dos capturas de pantalla tomadas durante la ejecución de la simulación en Unity.

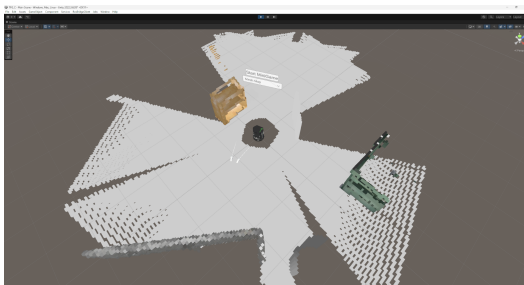


Figura 4.5: *
Vista en simulación virtual (A)

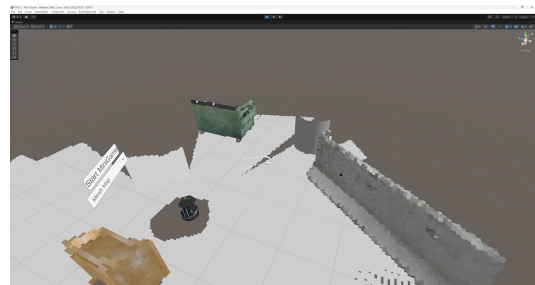


Figura 4.6: *
Vista en simulación virtual (B)

Figura 4.7: Capturas del entorno virtual recreado dinámicamente mediante mallas poligonales (Mesh Map).

4.5.3. Particle Map

En lugar de generar geometría sólida, este modo representa cada vóxel como una partícula del sistema de partículas de Unity, con posición, color y tamaño fijos, sin simulación física. El sistema de partículas está muy optimizado para manejar grandes cantidades de elementos simples, lo que hace que este modo sea más eficiente que el *Mesh Map* para mapas muy densos.

La optimización más relevante es la actualización incremental, en lugar de reconstruir el array completo en cada actualización, solo se inicializan las partículas nuevas desde la última posición conocida. Una vez que el mapa deja de recibir puntos nuevos, el coste de actualización por *frame* es prácticamente nulo.

A diferencia del uso de mallas conectadas, la aproximación mediante mapas de partículas *Particle Map* renderiza los puntos del sensor de manera independiente en el espacio 3D. Esto conserva con alta fidelidad la naturaleza discreta de los datos procedentes de ROS (ver Figura 4.10).

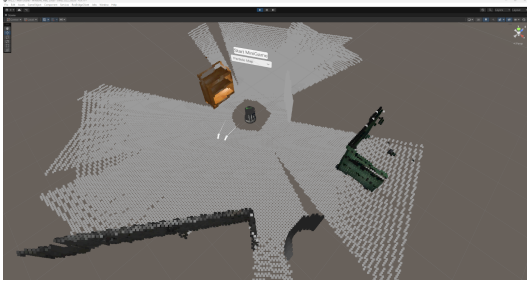


Figura 4.8: *

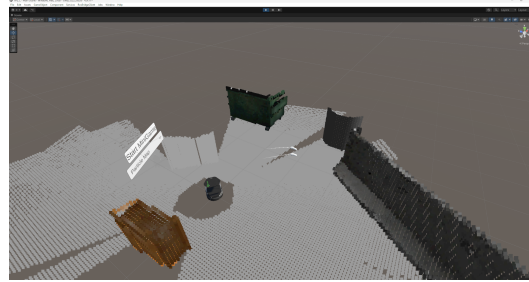


Figura 4.9: *

Densidad de partículas (Perspectiva 1) Densidad de partículas (Perspectiva 2)

Figura 4.10: Representación en Unity de la nube de puntos recibida mediante el sistema de mapas de partículas (Particle Map).

4.5.4. OctoMap

Este modo recibe los datos desde `octomap_server`, un nodo de ROS que construye y mantiene un mapa volumétrico de vóxeles a partir de la nube de puntos, aplicando internamente un proceso de fusión y filtrado de ruido. La nube que publica ya está en el marco de referencia del mapa, por lo que no requiere transformación TF en Unity.

La representación visual se basa en instanciado GPU, se renderizan hasta 1023 vóxeles en una única llamada a la GPU, repitiendo el proceso en lotes hasta cubrir el mapa completo. Esto permite representar cientos de miles de vóxeles con un

coste de renderizado muy bajo. El resultado visual es el más limpio de los cuatro modos, ya que los artefactos de ruido han sido filtrados por `octomap_server` antes de llegar a Unity.

La recepción del mensaje ocurre en el hilo de `rosbridge`. Para no bloquear ese hilo mientras se procesa, los puntos nuevos se acumulan en listas locales y se transfieren de forma atómica a unas listas protegidas por un mutex. El hilo principal de Unity lee esas listas durante cada *frame* y las aplica al mapa acumulativo.

Cuando el mensaje no incluye información de color, se puede asignar un color en función de la altura del vóxel interpolando sobre un gradiente configurable desde el inspector, lo que produce una visualización intuitiva incluso sin datos de color del sensor.

Por último, la representación mediante *Octo Map* discretiza el espacio tridimensional en cubos o vóxeles estructurados en un árbol octree. Como se observa en la Figura 4.13, esta técnica ofrece un excelente compromiso entre la resolución visual del entorno y el coste computacional.

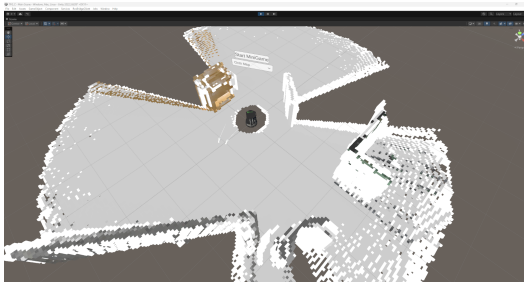


Figura 4.11: *

Estructura de vóxeles (Captura 1)

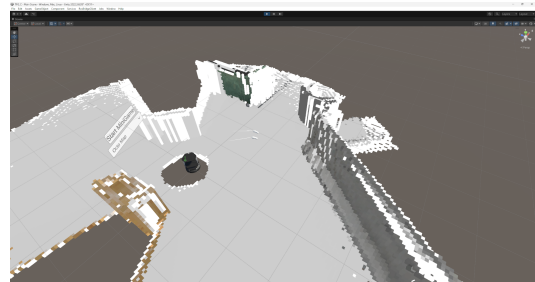


Figura 4.12: *

Estructura de vóxeles (Captura 2)

Figura 4.13: Reconstrucción tridimensional volumétrica basada en la técnica OctoMap dentro del motor Unity.

4.5.5. Comparativa de representaciones

La Tabla 4.1 resume las características principales de cada modo.

Característica	Wall Map	Mesh Map	Particle Map	OctoMap
Fuente de datos	Mapa 2D gmapping	Nube puntos RGB-D	Nube puntos RGB-D	octomap_server
Geometría	Plana (2D extruido)	Sólida 3D	Partículas	Vóxeles 3D
Color	No	Sí	Sí	Sí (o por altura)
Rendimiento	Muy alto	Moderado	Alto	Alto
Calidad visual	Básica	Alta	Alta	Alta
Ruido del sensor	No afecta	Visible	Visible	Filtrado
Uso recomendado	Visión rápida	Reconstrucción detallada	Mapas densos	Representación limpia

Tabla 4.1: Comparativa de los cuatro modos de representación del entorno.

4.5.6. Gestor de mapas

El componente `Map3DManager` actúa como punto de entrada único para la selección de representación. Cuando el usuario cambia el mapa desde la interfaz, este gestor activa el objeto del mapa elegido y desactiva los demás. Mantener los cuatro objetos en memoria simultáneamente, aunque solo uno esté visible, permite cambiar entre representaciones de forma instantánea y sin tiempos de carga.

4.6. Persistencia de mapas

Construir un mapa 3D desde cero requiere que el robot explore el entorno durante varios minutos. Para evitar repetir ese proceso en cada sesión, el sistema incluye un gestor de persistencia que serializa y deserializa cada tipo de mapa en disco usando JSON.

El formato JSON se eligió por simplicidad e interoperabilidad, ya que en un contexto académico, poder abrir el archivo con un editor de texto y depurar su contenido tiene más valor que el ahorro de espacio de un formato binario. Además, estos mapas en ninguna de las pruebas realizadas, ninguno de los mapas ha llegado a superar lo 100MB de espacio.

Los datos guardados varían según el tipo de mapa. Para el *Wall Map* se

serializan la posición, rotación y escala de cada cubo activo. Adicionalmente, también existe la opción de guardar el *Wall Map* como un *prefab*, útil para reutilizar el mapa como *asset* estático. Para el *Mesh Map* y el *Particle Map* se almacenan las posiciones y colores de los vóxeles. Para el *Octo Map* se guardan las posiciones como coordenadas de mundo flotantes, no como claves cuantizadas, de forma que el archivo sea independiente del tamaño de vóxel configurado en el momento del guardado.

Una funcionalidad especialmente útil es el modo sin conexión, cuando el *flag* de conexión a ROS está desactivado, el sistema puede cargar y visualizar un mapa previamente guardado sin necesitar ningún robot conectado. Esto permite usar el sistema para demostraciones o probar el minijuego sin acceso al *hardware*.

4.7. Interacción y teleoperación

4.7.1. Entorno VR y jugador

El entorno virtual está construido con el paquete XR Interaction Toolkit de Unity, que proporciona los componentes necesarios para el seguimiento de cabeza y manos, la locomoción y la interacción con elementos de la escena. El jugador puede moverse por el mapa generado, explorar el entorno desde distintas perspectivas y participar en el minijuego.

Para mitigar el mareo por movimiento durante el desplazamiento, se aplica un efecto de viñeta que oscurece los bordes del campo de visión al moverse, una técnica estándar en experiencias de VR con movimiento artificial.

Cabe añadir que se ha implementado una ligera interfaz de usuario es un panel flotante en el espacio virtual. Desde ella el usuario puede cambiar entre los cuatro tipos de representación del mapa mediante un desplegable e iniciar el minijuego, como se aprecia en la Figura 4.16.

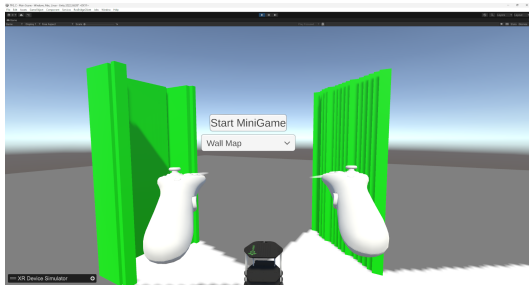


Figura 4.14: *
Menú plegado

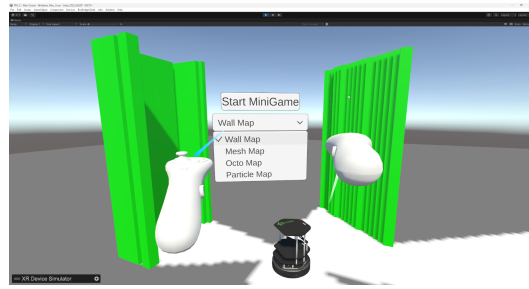


Figura 4.15: *
Menú desplegado

Figura 4.16: Interfaz de usuario integrada en el entorno inmersivo para la gestión de los mapas tridimensionales y el inicio del minijuego.

4.7.2. Sistema de puntos de vista

El sistema permite alternar entre tres perspectivas mediante un botón del mando, dando flexibilidad para adaptar la vista a cada momento de la experiencia.

La primera es la más obvia, la vista en primera persona del jugador, donde el usuario puede moverse libremente por el entorno virtual del mapa. La posición se guarda al salir de este modo y se restaura al volver.

Las otras dos perspectivas son las vistas en primera y tercera persona del robot, que desactivan la cámara del visor y activan una cámara virtual montada en el modelo del robot. La primera persona maximiza la inmersión, mientras que la tercera facilita la comprensión de la posición del robot en el entorno, siendo especialmente útil en espacios reducidos. Al entrar en cualquiera de estas dos vistas, el control del movimiento pasa del jugador a automáticamente al robot.

Al cambiar de perspectiva, todas las cámaras se desactivan explícitamente antes de activar la nueva, para evitar que dos cámaras estén activas al mismo tiempo.

4.7.3. Teleoperación del robot

El robot puede controlarse desde las vistas en primera y tercera persona usando el joystick del mando de VR. El eje vertical se traduce en velocidad lineal y el horizontal en velocidad angular, con factores de escala configurables desde la interfaz. Los comandos se publican en el topic `/cmd_vel` del robot a 5 Hz, frecuencia suficiente para una respuesta fluida sin saturar el canal de comunicación.

El minijuego puede bloquear el movimiento del robot en momentos concretos

de su lógica, como durante el nivel 4 mientras el robot no ha recibido la energía del jugador. En ese caso, aunque el usuario mueva el joystick, los comandos enviados son siempre cero.

4.8. Minijuego cooperativo

El minijuego, a pesar de su sencillez, aprovecha el espacio 3D generado para crear una actividad estructurada con objetivos claros. Está diseñado para que el usuario con las gafas de VR consiga coordinar y mover por el espacio a su propio personaje y al robot, haciendo que trabajen juntos para completar cuatro niveles de dificultad progresiva, siguiendo el esquema de una máquina de estados donde cada estado define qué objetos están activos en la escena y qué condiciones deben cumplirse para avanzar.

Los objetivos de cada nivel se generan en posiciones aleatorias del mapa que sean celdas libres, consultando el suscriptor del mapa de ocupación. El sistema, de forma aleatoria, busca una celda libre que esté además a una distancia mínima del origen, para evitar que los objetivos aparezcan demasiado cerca del punto de partida.

Nivel 1. Navegación del robot: se genera un objetivo cercano al robot. El nivel se completa cuando el robot lo alcanza. Sirve como introducción al control del robot desde VR.

Nivel 2. Navegación del jugador: se genera un objetivo cercano al jugador. El nivel se completa cuando el jugador se desplaza hasta él. Introduce el movimiento del jugador en el entorno virtual.

Nivel 3. Sincronización: se generan dos objetivos, uno para cada agente. El nivel se completa cuando ambos están en sus respectivos objetivos durante dos segundos consecutivos. Si alguno sale antes de tiempo, el contador se reinicia. Introduce la coordinación entre jugador y robot.

Nivel 4. Transferencia de energía: tiene tres fases. Primero, el jugador recoge un objeto de energía que aparece cerca de él. Luego, se aproxima al robot para transferírselo. Finalmente, el robot, que hasta ese momento no puede moverse, debe llegar a un objetivo en el mapa. La Figura 4.17 ilustra los *assets* generados durante esta mecánica cooperativa.

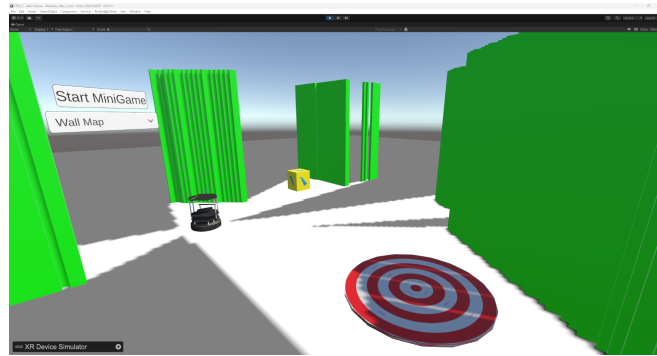


Figura 4.17: Elementos visuales correspondientes a los objetivos y zonas de transferencia de energía en el nivel 4 del minijuego cooperativo.

Al completar el nivel 4, el juego vuelve al estado inicial y todos los objetos del nivel se eliminan de la escena. Durante el desarrollo se añadió además una tecla de depuración que salta directamente al siguiente nivel, para facilitar la depuración durante el desarrollo.

4.9. Problemas encontrados durante el desarrollo

4.9.1. Obsolescencia de la plataforma.

El Turtlebot2 ejecuta Ubuntu 16.04, ROS Kinetic y Gazebo 7, todos con el soporte oficial terminado desde 2021. Esto se tradujo en paquetes de Python incompatibles con versiones modernas, dependencias ausentes en los repositorios, documentación desactualizada y comportamientos distintos entre lo que dice la documentación y lo que hace el *software* real. Fue la fuente de problemas más persistente durante todo el desarrollo y supuso una pérdida de tiempo considerable.

4.9.2. Incidencia en RosSharp.

Durante el proceso de integración de RosSharp 2.1.0 se detectó un problema de compilación en el archivo `GetParamRequest.cs`. El origen del error se encontraba en la instrucción `this.@default = default;`, ya que `default` es una palabra reservada del lenguaje C# y, en las versiones del compilador empleadas por Unity, no podía utilizarse de esa forma dentro de la asignación. Para resolver la incidencia, la expresión fue reemplazada por `this.@default = @default;`, permitiendo

asignar correctamente el argumento recibido por el constructor. Aunque la modificación necesaria fue mínima, este caso pone de manifiesto el estado de algunas bibliotecas del entorno.

4.9.3. Condiciones de carrera entre hilos.

Los datos de ROS llegan en hilos secundarios y no pueden aplicarse directamente a la escena de Unity. En los primeros diseños, el acceso sin protección a las estructuras compartidas causaba excepciones al modificar una colección mientras se iteraba desde otro hilo. La solución fue proteger todos los accesos con bloqueos, realizando las operaciones costosas fuera del bloque crítico copiando primero los datos a una estructura local.

4.9.4. Acumulación ilimitada de vóxeles.

Sin ningún límite, el diccionario de vóxeles crecía hasta agotar la memoria en sesiones largas. Se añadió un límite máximo configurable en todos los receptores, al alcanzarlo, los puntos nuevos se descartan. No es la solución más sofisticada, pero es sencilla y suficiente para sesiones de duración razonable.

4.9.5. Desalineaciones en la reconstrucción tridimensional

Durante las pruebas se observaron pequeñas duplicaciones de paredes y objetos en las reconstrucciones generadas a partir de las nubes de puntos. Estas desalineaciones, normalmente de unos pocos centímetros, se debían a la combinación de varios factores, errores acumulados en la odometría, correcciones realizadas por SLAM, desfases temporales entre las nubes de puntos y las transformadas TF, movimiento del robot durante la captura y el propio ruido del sensor RGB-D. Para reducir este problema se implementó la interpolación temporal de transformadas y se limitó la incorporación de nuevas capturas a periodos en los que el robot permanecía inmóvil. Aun así, debido a las limitaciones inherentes de los sensores y de la localización, no fue posible eliminar completamente estos artefactos, aunque si reducirlos en gran medida.

4.9.6. Acceso limitado al robot físico.

Durante casi todo el desarrollo las pruebas se realizaron sobre el simulador, con acceso al robot real solo en la fase final. Varias funcionalidades que funcionaban perfectamente en Gazebo fallaron sobre el *hardware*, *topics* con nombres distintos, latencias de red ausentes en el entorno de desarrollo, diferencias en

el comportamiento del driver de Kobuki y configuración errónea de la cámara Orbbec Astra. Depurar estos problemas sobre el robot es considerablemente más lento que hacerlo en el simulador.

4.9.7. Condiciones en el entorno real.

Las pruebas presenciales se realizaron en un laboratorio en una planta sótano con cobertura reducida, usando un teléfono móvil como punto de acceso WiFi. Esto limitó el ancho de banda y generó picos de latencia que obligaron a reducir la densidad de los mapas y la frecuencia de actualización respecto a los parámetros del simulador. Otro factor limitante fue el equipo utilizado durante las pruebas. Mientras que el entorno de desarrollo se ejecutaba en un ordenador de sobremesa con un procesador Ryzen 7 7700, una tarjeta gráfica RTX 5060 Ti de 16 GB y 32 GB de memoria RAM DDR5, las pruebas se realizaron en un portátil equipado con un Intel i5-7300HQ, una GTX 1050 Mobile y 8 GB de RAM DDR4. En términos aproximados, el ordenador de desarrollo ofrece alrededor de seis veces más rendimiento de CPU, cinco veces más rendimiento gráfico y cuatro veces más memoria disponible, lo que condicionó significativamente el rendimiento observado durante las pruebas presenciales.

5

Experimentos y métricas

En este capítulo se describen las pruebas realizadas para evaluar el correcto funcionamiento y el rendimiento del sistema desarrollado. El objetivo es analizar de forma objetiva aspectos como el coste computacional de cada mapa, la fidelidad de la reconstrucción, el comportamiento del sistema de guardado y carga y la jugabilidad del minijuego.

5.1. Metodología experimental

Las pruebas se realizaron en dos contextos diferenciados.

- **Entorno simulado**, en el que el Turtlebot2 estaba ejecutado en Gazebo sobre una máquina virtual Ubuntu, conectada a Unity mediante `rosbridge`. Este entorno permitió condiciones de red perfectas y reproducibilidad total de los experimentos. La Figura 5.1 ilustra las perspectivas de este entorno virtual controlado.
- **Entorno real**, con el Turtlebot2 físico en el laboratorio, con un dispositivo móvil como punto de acceso WiFi y utilizando un portátil, que además era menos potente que el PC de desarrollo. Este entorno, teniendo en cuenta además que el laboratorio estaba en una planta sótano con baja cobertura, introdujo variabilidad de red y un menor rendimiento.

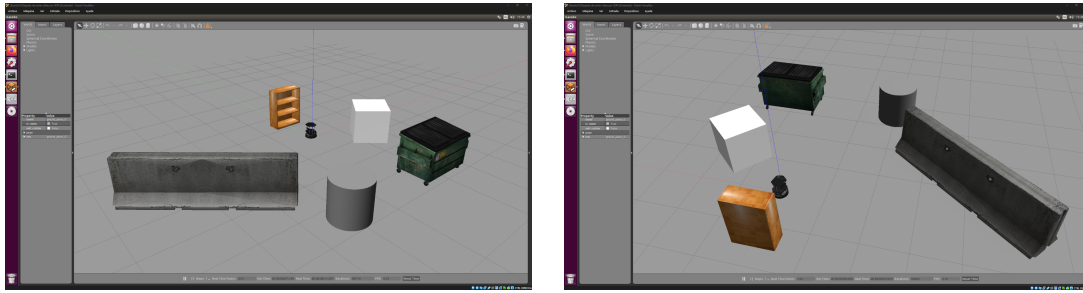


Figura 5.1: Entorno de simulación en Gazebo utilizado para la captura de métricas bajo condiciones de red y entorno controladas.

En ambos entornos se ejecutaron las siguientes pruebas:

- Medición de los FPS de Unity con cada tipo de mapa activo durante la exploración.
- Monitorización del uso de CPU y memoria durante la reconstrucción de mapas.
- Evaluación de la fidelidad visual de cada representación 3D respecto al entorno real.
- Ciclo completo de guardado y carga para cada tipo de mapa.
- Verificación nivel a nivel del minijuego.

5.2. Parámetros de configuración del sistema

La siguiente tabla recoge los parámetros utilizados durante las pruebas. En el entorno real se aplicaron configuraciones más conservadoras en cuanto a densidad de los mapas y frecuencia de actualización, condicionadas por el ancho de banda disponible y las capacidades del portátil de pruebas. En el entorno simulado se exploraron los límites del sistema con parámetros más exigentes.

Parámetro	Alta calidad	Rendimiento reducido
Tamaño de vóxel	0,05 m	0,10 m
Máximo de vóxeles	200.000	50.000
Point skip en Unity	1	5
Decimación nube (pointcloud_relay)	skip=10	skip=15
Resolución octomap_server	0,05 m	0,10 m
Frecuencia máxima de nube decimada	2 Hz	2 Hz
Intervalo de reconstrucción de malla	1,0 s	1,0 s
Cubos por frame (reconstrucción incremental)	500	500
Tiempo quieto para captura (settleTime)	1,0 s	1,5 s

Tabla 5.1: Parámetros de configuración del módulo de mapas.

5.3. Resultados

5.3.1. Rendimiento por tipo de mapa

La manera principal para evaluar el rendimiento de cada representación son los FPS de Unity, que debe mantenerse por encima de 90 FPS para garantizar una experiencia de realidad virtual cómoda sin mareos.

Una observación importante y común a todos los modos de mapa es que el coste computacional significativo no proviene de usar el mapa, sino de construirlo. Cuando el mapa está ya completo, se puede usar e interactuar con un rendimiento estable en todos los casos. Los bajones de FPS descritos a continuación ocurren únicamente mientras el robot está explorando y llegan datos nuevos que requieren actualizar la representación.

El mapa de paredes mantuvo unos FPS estables por encima de 90 FPS en todo momento, tanto en el simulador como en el entorno real. Al estar basado en cubos estáticos con un material compartido, el coste de renderizado es fijo una vez construido el mapa y no crece durante la exploración. La reconstrucción del mapa al recibir una actualización del tópico `/map` produce un breve pico de CPU durante la devolución y reasignación de cubos del *pool*, aunque con el *pool* preestablecido su impacto es imperceptible en los FPS.

El mapa de partículas se mantuvo igualmente por encima de 90 FPS con hasta 50.000 partículas activas. El sistema de partículas de Unity gestiona de forma interna el renderizado en la GPU de forma eficiente, y la actualización incremental garantiza que el coste de CPU por *frame* sea mínimo una vez que el mapa deja de recibir puntos nuevos. El pico de inicialización al arrancar, cuando

las primeras nubes de puntos comienzan a llenar el mapa, es el momento de mayor carga, aunque no llega a producir caídas de *framerate* perceptibles.

El mapa de malla fue el modo con mayor variabilidad de rendimiento. Durante las fases de reconstrucción de la malla se apreciaba una ligera caída de FPS de aproximadamente 10-20 FPS, correspondiente al procesamiento del *chunk* de cubos por *frame* de la corrutina. Esta variabilidad desaparece una vez que el mapa está completamente construido y no llegan puntos nuevos. Con la reconstrucción síncrona original, los picos llegaban a causar caídas de más de 50 FPS durante varios *frames* consecutivos, haciendo la experiencia VR completamente inaceptable. La versión con corrutina redujo este impacto a niveles tolerables.

El mapa de octree se mantuvo por encima de 90 FPS con hasta 100.000 vóxeles instanciados, gracias al uso de `DrawMeshInstanced`. El coste de la reconstrucción del cache de matrices, que solo ocurre cuando el mapa cambia, es lineal con el número de vóxeles pero se distribuye en un único *frame* y su duración es inferior a 2 ms para los tamaños de mapa probados.

5.3.2. Fidelidad de la reconstrucción

La fidelidad visual de cada representación se evaluó haciendo una comparación visual del mapa generado con el entorno real del laboratorio o del mundo virtual de Gazebo, identificando si las paredes, obstáculos y objetos principales eran reconocibles. La Figura 5.2 muestra la misma zona del entorno físico capturada fotográficamente y la figura 5.7 muestra el resultado procesado por las cuatro técnicas de reconstrucción abordadas en el estudio.



Figura 5.2: Fotografía real del laboratorio utilizado para las pruebas.

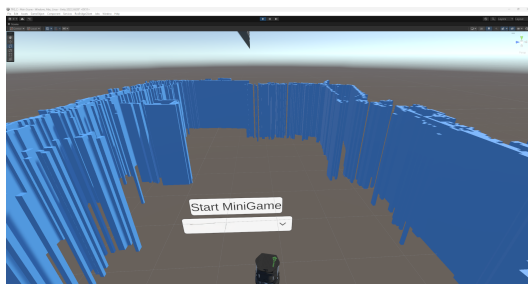


Figura 5.3: *
Wall Map

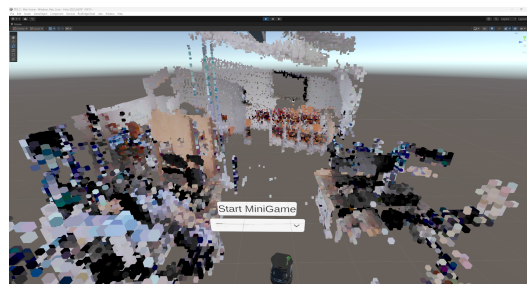


Figura 5.4: *
Mesh Map

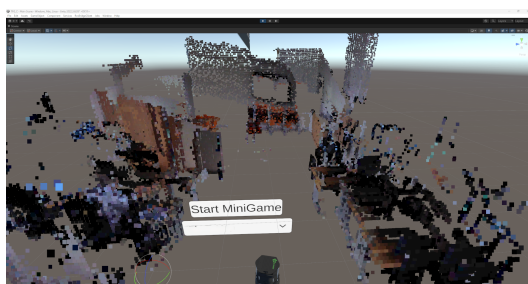


Figura 5.5: *
Particle Map

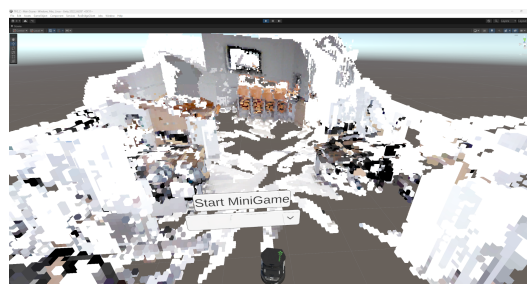


Figura 5.6: *
OctoMap

Figura 5.7: Comparativa directa de la reconstrucción tridimensional. Representación visual generada por los cuatro modelos implementados.

El *Wall Map* reprodujo correctamente la estructura principal del entorno (paredes perimetrales y obstáculos grandes), con una fidelidad directamente proporcional a la calidad del mapa de gmapping. Las zonas no exploradas por el robot aparecen como áreas vacías sin paredes, lo cual es coherente con la información disponible.

El *Particle Map* y el *Mesh Map* produjeron representaciones tridimensionales más detalladas, capturando no solo las paredes sino partes del suelo, superficies de mesas y otros objetos. La densidad de la nube es perceptiblemente menor en el entorno real respecto al simulado, debido a las limitaciones del sensor Orbbec Astra y a la decimación más agresiva necesaria por la red. En zonas bien iluminadas y con superficies planas, la reconstrucción fue suficientemente detallada para reconocer la estructura del entorno. Aunque también es importante mencionar que en ocasiones ocurrían duplicados desplazados de un mismo elemento, debido a pequeñas desviaciones en la odometría.

El *Octo Map* produjo la representación más ordenada y geoméricamente limpia, ya que `octomap_server` aplica internamente un proceso de fusión y filtrado

de ruido sobre la nube acumulada. Esto resulta en un mapa con menos artefactos que el *Particle Map* o el *Mesh Map*, a costa de que la actualización es menos frecuente.

5.3.3. Guardado y carga de mapas

Se realizaron pruebas de ciclo completo para los cuatro tipos de mapa, guardando, reiniciando Unity y cargando. En todos los casos los mapas se restauraron correctamente, con posiciones y colores de vóxeles idénticos a los originales. El tiempo de guardado y carga fue proporcional al número de elementos almacenados, pero aún así tardando siempre menos de 10 segundos en los mapas más densos. Ya que estos tiempos no se realizan durante la exploración activa, si no al final de la exploración y al principio de la ejecución, es más que aceptable el tiempo de espera.

5.3.4. Minijuego

Se completó el minijuego en el entorno simulado y en el entorno real numerosas veces, siempre sin *bugs* y con tiempos consistentes. En los tres primeros niveles la mecánica solo consiste en mover al jugador o al robot, y es en el cuarto nivel donde se introduce una nueva mecánica, las baterías, que se traspasan del jugador al robot por cercanía, funcionaron en todas las ocasiones sin problema. Además, la generación de objetivos en las celdas libres funcionó sin incidencias en todos los casos, no apareciendo ningún objetivo en una zona inaccesible ni en zonas desconocidas.

6

Conclusiones y trabajos futuros

En este capítulo final se presenta una valoración global del trabajo realizado, analizando el grado de cumplimiento de los objetivos planteados, los resultados obtenidos y las principales aportaciones del sistema desarrollado. Asimismo, se identifican las limitaciones del sistema y se proponen líneas de trabajo futuras.

6.1. Discusión de resultados

Los resultados obtenidos permiten extraer conclusiones diferenciadas según el componente evaluado.

En cuanto a la representación del entorno, el sistema demostró ser capaz de ofrecer cuatro modalidades de visualización del entorno funcionales y complementarias entre sí, cada una con sus propias fortalezas. El *Wall Map* es el más eficiente y estable, adecuado como representación por defecto. El *Octo Map* ofrece la mayor calidad visual con bajo coste de renderizado gracias al instanciado GPU, siendo la mejor opción. El *Particle Map* y el *Mesh Map* son más adecuados para visualizaciones densas en tiempo real, aunque a costa de mayor variabilidad en el rendimiento.

Respecto al sistema de guardado y carga, los tiempos de serialización son aceptables para el uso previsto, que no es en tiempo real sino como operación puntual al inicio o al final de una sesión. La capacidad de trabajar en modo *offline* con mapas preconstruidos resulta especialmente valiosa para demostraciones,

entrenamiento o videojuegos, sin depender de la disponibilidad del robot.

El minijuego demostró ser un ejemplo eficaz de las posibilidades que ofrece una experiencia de juego utilizando mapas reales, mediante tareas con objetivos claros y una progresión gradual de la dificultad. La introducción escalonada de los distintos conceptos, comenzando por la navegación individual, el movimiento del jugador, el cambio de perspectiva para controlar y manejar al robot, la cooperación entre ambos personajes y la transferencia de recursos, siguió una curva de aprendizaje natural que facilitó la familiarización del usuario con las mecánicas del sistema.

Además, los niveles evaluados se completaron de forma consistente durante las pruebas, lo que confirma la robustez tanto de la lógica implementada mediante la máquina de estados como del sistema de generación de objetivos dentro del mapa.

6.2. Cumplimiento de objetivos

A continuación se analiza el grado de cumplimiento de los objetivos definidos al inicio del trabajo.

6.2.1. Objetivo general

El objetivo general del proyecto, consistente en el desarrollo de un sistema que genere automáticamente un entorno de realidad virtual navegable a partir de la información sensorial del Turtlebot2 e integre sobre él una experiencia de juego interactiva, ha sido alcanzado satisfactoriamente. El sistema implementado es capaz de reconstruir la geometría del entorno físico en tiempo real, representarla mediante distintas técnicas de visualización tridimensional en Unity, e incorporar sobre ese entorno un jugador con controles de VR, un robot virtual sincronizado con el físico y un minijuego jugable que ilustra las posibilidades de interacción entre el jugador y el robot dentro del espacio construido.

6.2.2. Objetivos específicos

- **Integración del mapa de ocupación 2D en Unity:** se ha implementado el suscriptor del mensaje `nav_msgs/OccupancyGrid` en Unity, generando a partir de él una representación navegable del entorno como conjunto de paredes tridimensionales sobre las que el jugador y el robot virtual pueden moverse.

- **Múltiples técnicas de reconstrucción tridimensional:** se han implementado y comparado cuatro representaciones del entorno a partir de la nube de puntos RGB-D del robot, el *Wall Map* basado en el mapa de ocupación 2D (ya mencionado), el *Mesh Map* mediante malla poligonal procedural, el *Particle Map* mediante el sistema de partículas de Unity y el *Octo Map* mediante vóxeles. Cada una ofrece un compromiso diferente entre calidad visual y rendimiento, y pueden alternarse en tiempo de ejecución.
- **Integración del robot virtual:** se ha incorporado el modelo URDF del Turtlebot2 en el entorno virtual gracias al paquete ROS#, sincronizando su posición y orientación con la odometría publicada por el robot físico mediante el árbol de transformaciones TF de ROS, de forma que el robot virtual refleja en todo momento el movimiento del robot real, pudiendo además ser controlado con el mando de VR.
- **Minijuego cooperativo:** se han diseñado e implementado cuatro niveles de un minijuego de exploración en el que el jugador y el robot deben coordinarse para alcanzar objetivos generados en posiciones libres del mapa. Las mecánicas se introducen de forma progresiva y el juego es funcional tanto con el robot físico como con el robot virtual en el simulador.
- **Validación del sistema:** se han realizado pruebas funcionales en el entorno simulado con Gazebo y, de forma limitada, sobre el robot físico, evaluando el rendimiento gráfico, la coherencia espacial del mapa generado y la jugabilidad del minijuego, todas ellas dando resultados satisfactorios.

6.3. Conclusiones generales

El desarrollo de este trabajo ha permitido validar la viabilidad de generar automáticamente un entorno de realidad virtual navegable a partir de los datos sensoriales de un robot móvil de bajo coste, utilizando exclusivamente herramientas de acceso libre. La integración entre ROS, Unity y un visor de realidad virtual ha demostrado ser una combinación funcional y suficientemente eficiente para construir representaciones tridimensionales del entorno en tiempo real e incorporar sobre ellas una experiencia interactiva jugable.

Las cuatro representaciones del entorno implementadas, *WallMap*, *MeshMap*, *ParticleMap* y *OctoMap*, ofrecen compromisos distintos entre calidad visual y rendimiento, lo que permite adaptar el sistema a las capacidades del *hardware* disponible. En el entorno de desarrollo, con *hardware* moderno y potente, todas las representaciones se mantuvieron por encima de los 90 FPS recomendados para realidad virtual. En el entorno de pruebas con el robot real, las limitaciones de red introducidas por el uso de un dispositivo móvil como punto de acceso en un espacio con cobertura reducida, junto con las capacidades más modestas del

portátil utilizado, obligaron a operar con parámetros más conservadores, aunque el sistema siguió siendo funcional en ambos casos.

La antigüedad del *stack* de *software* del Turtlebot2, con Ubuntu 16.04 y ROS Kinetic, supuso la limitación más persistente durante el desarrollo, con problemas de compatibilidad, dependencias abandonadas y documentación desactualizada. A pesar de ello, el sistema desarrollado demuestra que es posible construir sobre esta plataforma una experiencia inmersiva y gamificada sin necesidad de actualizarla o sustituirla.

El minijuego implementado, aunque sencillo, ilustra de forma concreta las posibilidades de combinar un mapa generado desde datos reales con mecánicas de juego, tanto con el robot físico como con mapas previamente guardados y el robot virtual. La arquitectura modular adoptada facilita además la incorporación de nuevas representaciones, mecánicas o dispositivos sin necesidad de rediseñar el sistema.

En conjunto, el proyecto demuestra que ámbitos que parecen lejanos como la captura de datos en robótica y los videojuegos, pueden ser en realidad más cercanos y existen formas de utilizar ambos para construir experiencias interactivas inmersivas con un coste de desarrollo accesible con plataformas robóticas de bajo coste, todo para un contexto académico.

6.4. Trabajos futuros y posibles mejoras

A partir del sistema desarrollado se identifican varias líneas de trabajo que permitirían ampliar y mejorar sus capacidades:

- **Actualizar el archivo de guardado dinámicamente**, para en el caso de que se interrumpa la sesión de forma inesperada, no se pierda el progreso, haciendo guardados incrementales cada cierta cantidad de vóxeles nuevos.
- **Fusionar representaciones complementarias**, aprovechando la estructura del *Wall Map* y las texturas captadas por el *Particle Map*, generando así un mapa sencillo y ligero pero aún así a color y con profundidad.
- La principal línea de trabajo futuro es utilizar este sistema para **crear videojuegos más complejos** y que aprovechen en mayor medida la creación de una replica digital del espacio real. Ejemplos de esto podrían ser juegos colaborativos en el que un usuario maneja al robot y el otro a un avatar virtual, juegos que exploren mecánicas más avanzadas para la creación de puzzles, o virar a una dirección totalmente distinta introduciendo la mecánica de disparos en un videojuego al estilo arena.

- **Mejorar la calidad y rendimiento de las nubes de puntos**, para así lograr mapas más grandes y detallados sin comprometer el sistema.
- **La adopción de un robot de nueva generación**, como Turtlebot4 u otras plataformas equivalentes, supondría una mejora significativa en el sistema al proporcionar tanto hardware más moderno y potente, incluyendo en sensor de profundidad, como la capacidad de utilizar software moderno y actual, siendo compatible con herramientas más avanzadas y permitiendo su sostenibilidad a largo plazo.

Estas líneas de trabajo abrirían el sistema hacia aplicaciones más completas y versátiles, pudiendo aprovechar tanto la interacción persona-robot como los entornos virtuales inmersivos.

Bibliografía

- [1] Tomas Akenine-Möller, Eric Haines, and Naty Hoffman. *Real-Time Rendering*. CRC Press, 4th edition, 2018.
- [2] Nunzio Barone, Walter Brescia, Gabriele Santangelo, Antonio Pio Maggio, Ivan Cisternino, Luca De Cicco, and Saverio Mascolo. Real-time point cloud transmission for immersive teleoperation of autonomous mobile robots. *Proceedings of the 16th ACM Multimedia Systems Conference (MMSys '25)*, pages 311–316, 2025. doi: 10.1145/3712676.3719263. URL <https://dl.acm.org/doi/10.1145/3712676.3719263>.
- [3] Mafkereseb Kassahun Bekele, Roberto Pierdicca, Emanuele Frontoni, Eva Savina Malinverni, and James Gain. A survey of augmented, virtual, and mixed reality for cultural heritage. *Journal on Computing and Cultural Heritage*, 11(2):7:1–7:36, 2018. doi: 10.1145/3145534.
- [4] Corneliu Boje, Annie Guerriero, Sylvain Kubicki, and Yacine Rezgui. Towards a semantic construction digital twin: Directions for future research. *Automation in Construction*, 114:103179, 2020. doi: 10.1016/j.autcon.2020.103179.
- [5] Robert Frank Codd-Downey, Parisa Mojiri Forooshani, Andrew Speers, Hui Wang, and Michael Jenkin. From ros to unity: Leveraging robot and virtual environment middleware for immersive teleoperation. In *2014 IEEE International Conference on Information and Automation (ICIA)*, pages 932–936, 2014. doi: 10.1109/ICInfA.2014.6932785. URL https://www.researchgate.net/publication/286716293_From_ROS_to_unity_Leveraging_robot_and_virtual_environment_middleware_for_immersive_teleoperation.
- [6] Randima Fernando, editor. *GPU Gems*. Addison-Wesley, 2004. Especially chapters on geometry and real-time rendering techniques.
- [7] Peter Henry, Michael Krainin, Evan Herbst, Xiaofeng Ren, and Dieter Fox. Rgb-d mapping: Using kinect-style depth cameras for dense 3d modeling of indoor environments. *The International Journal of Robotics Research*, 31(5):647–663, 2012. doi: 10.1177/0278364911434148. URL https://www.researchgate.net/publication/254098812_RGB-D_Mapping_Using_Kinect-Style_Depth_Cameras_for_Dense_3D_Modeling_of_Indoor_Environments.

- [8] Matheus Nicolás Hernández, Brenno Domingues, Roberto Simoni, Douglas Negri, Diego De Souza, and Luís Gonzaga Trabasso. Robot teleoperation via virtual reality: A unity and ros approach. In *2024 3rd International Conference on Automation, Robotics and Computer Engineering (ICARCE)*, 2024. doi: 10.1109/ICARCE63054.2024.00012. URL https://www.researchgate.net/publication/391389407_Robot_Teleoperation_via_Virtual_Reality_A_Unity_and_ROS_Approach.
- [9] Armin Hornung, Kai M. Wurm, Maren Bennewitz, Cyrill Stachniss, and Wolfram Burgard. Octomap: An efficient probabilistic 3d mapping framework based on octrees. *Autonomous Robots*, 34(3):189–206, 2013. doi: 10.1007/s10514-012-9321-0. URL <https://www.arminhornung.de/Research/pub/hornung13auro.pdf>.
- [10] Ke Li, Reinhard Bacher, Susanne Schmidt, Wim Leemans, and Frank Steinicke. Reality fusion: Robust real-time immersive mobile robot teleoperation with volumetric visual data fusion. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2024. URL https://www.edit.fis.uni-hamburg.de/ws/files/55138101/Camera_Ready.pdf. Github: url = <https://github.com/uhhhci/RealityFusion>.
- [11] Microsoft. Garbage collection fundamentals. <https://learn.microsoft.com/en-us/dotnet/standard/garbage-collection/fundamentals>, 2024. Accessed: 2026-06-14.
- [12] Robert Nystrom. *Game Programming Patterns*. Genever Benning, 2014. URL <https://gameprogrammingpatterns.com/>.
- [13] Fabio Remondino, Maria Grazia Spera, Erica Nocerino, Fabio Menna, and Francesco Nex. State of the art in high density image matching. In *The Photogrammetric Record*, volume 29, pages 144–166, 2014. doi: 10.1111/phor.12063.
- [14] Markus Schütz. Potree: Rendering large point clouds in web browsers. In *Proceedings of Eurographics Symposium on Rendering*. The Eurographics Association, 2016.
- [15] Unity Technologies. *Unity Robotics Hub*, 2024. URL <https://github.com/Unity-Technologies/Unity-Robotics-Hub>.
- [16] Unity Technologies. Coroutines. <https://docs.unity3d.com/Manual/Coroutines.html>, 2024. Accessed: 2026-06-14.
- [17] Unity Technologies. Gpu instancing. <https://docs.unity3d.com/Manual/GPUInstancing.html>, 2024. Accessed: 2026-06-14.
- [18] Unity Technologies. Mesh api. <https://docs.unity3d.com/ScriptReference/Mesh.html>, 2024. Accessed: 2026-06-14.
- [19] Unity Technologies. Particle system. <https://docs.unity3d.com/Manual/ParticleSystems.html>, 2024. Accessed: 2026-06-14.

- [20] George Vosselman and Hans-Gerd Maas. *Airborne and Terrestrial Laser Scanning*. Whittles Publishing, 2010.
- [21] David Whitney, Eric Rosen, Daniel Ullman, Elizabeth Phillips, and Stefanie Tellex. Ros reality: A virtual reality framework using consumer-grade hardware for ros-enabled robots. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2018. URL https://www.researchgate.net/publication/326693689_ROS_Reality_A_Virtual_Reality_Framework_Using_Consumer-Grade_Hardware_for_ROS-Enabled_Robots.
- [22] Graham Wihlidal. Rendering the world of Star Wars Battlefront. In *Game Developers Conference (GDC)*, 2017. URL <https://www.gdcvault.com/play/1024087>.
- [23] Georgios N. Yannakakis and Julian Togelius. *Artificial Intelligence and Games*. Springer, 2018. doi: 10.1007/978-3-319-63519-4.
- [24] Karim Youssef, Sherif Said, Samer Al Kork, and Taha Beyrouthy. Telepresence in the recent literature with a focus on robotic platforms, applications and challenges. *Robotics*, 12(4):111, 2023. doi: 10.3390/robotics12040111. URL <https://www.mdpi.com/2218-6581/12/4/111>.
- [25] Michael Zyda. From visual simulation to virtual reality to games. *Computer*, 38(9):25–32, 2005. doi: 10.1109/MC.2005.297.